REALIZATION CALCULUS

CHAPTER I

Projector-Resolved Seed Calculus I

Typed operator systems, seed rank, and exact packet coordinates

Komorebi P.

Computational companion

Computational records and interfaces

This computational companion represents Chapter I as a collection of mathematical records that can be inspected and queried, with a finite subset supported by executable interfaces. The statement inventory is formal/coverage/COVERAGE_108.json. The executable runtime has a separate source entry in the source register. The catalogue and runtime have the distinct scopes explained below.

The catalogue contains 108 mathematical records, including 69 theorem, proposition, corollary, and lemma records with source proofs. The formal correspondence is reported alongside each record. The preserved runtime interface has twenty typed signatures, and its separate Lean runtime map has eight mappings. Those inherited interfaces and receipts retain their original scope.

The mathematical source supplies the statements and proofs, including the split-semisimple hypotheses and explicit map qualifications. The supplementary worked models remain in the mathematical text. Stable identifiers connect the mathematical, formal, and computational material. Statement hashes and source locators distinguish the catalogue from the archived runtime payloads.

The verification record distinguishes evidence for the mathematical correspondence from the runtime receipts. Its source inventory and formal correspondence map are available under `formal/coverage/`, and compiler and replay receipts are under `formal/evidence/`. The Lean guide explains the formal sources, their hypotheses, and how to interpret those checks.

How to use this volume. Read Sections 1–3 for mathematical and evidence semantics. Sections 4–6 give the executable contracts and an exact worked example. Section 7 describes reproduction. Appendix A reproduces every canonical source statement and every attached proof body, with source locators and machine metadata. Later appendices give signatures, action coverage, operation vocabulary, and the runtime Lean map.

Contents

Computational records and interfaces **1**

1 The mathematical object behind the interface **4**
1.1 Operator systems, generated hulls, and seed rank . . . 4
1.2 Exact packet coordinates . . . 4
1.3 Transfer blocks and composition . . . 5
1.4 Intertwining and reconstruction . . . 5
1.5 The boundary of finite execution . . . 5

2 Canonical machine representation **5**
2.1 Entrypoint and package identity . . . 5
2.2 Record identity and source provenance . . . 6
2.3 Statement AST and typed signatures . . . 6
2.4 Dependencies and closure . . . 6
2.5 Operation profiles and actions . . . 7

3 Evidence semantics and certification scope **7**
3.1 Independent evidence coordinates . . . 7
3.2 Formal correspondence and checks . . . 7
3.3 Historical Lean linkage . . . 8
3.4 Recorded runtime checks . . . 8
3.5 What PASS means . . . 8

4 Executable action contracts **8**
4.1 Input and output conventions . . . 8
4.2 The six exact finite actions . . . 9
4.3 The two semantic gate actions . . . 9
4.4 Planner-only and fixture-only actions . . . 9

5 An exact worked execution **10**
5.1 Oblique packets and reconstruction . . . 10
5.2 Resolving and rebuilding an operator . . . 10
5.3 Composing resolved operators . . . 11
5.4 An intertwining computation . . . 11
5.5 Inspecting the dependency set . . . 12

6 Transport profiles in use **12**
6.1 Finite packet multiplication . . . 12
6.2 A countable result is blocked . . . 13
6.3 Changing a profile without changing evidence . . . 13

7 Reproducing the computational material **13**
7.1 Documented payload checks 14
7.2 Companion build 14
7.3 A source change is a new evidence task 14
7.4 Runtime interfaces and mathematical statements 14

A Mathematical record catalogue **15**
A.1 Foundations of seed generation 15
A.2 Packets and transfer graphs 42
A.3 Transport and realization 69

B Typed core signatures **82**

C Complete action registry **87**

D Operation and hypothesis vocabulary **93**

E Historical Lean record map **95**

F Preserved verification receipt **97**

1 The mathematical object behind the interface

1.1 Operator systems, generated hulls, and seed rank

A seed problem consists of an acting algebra, a module, and a family of seed vectors. The algebra must be fixed before the generation question is asked. For a unital algebra A acting on a module M and a finite seed family $S = \{s_1, \ldots, s_r\}$, the algebraic generated hull is

$$\mathrm{Hull}_A(S) = \sum_{j=1}^{r} As_j.$$

A one-seed system is cyclic. Seed rank measures the minimum cardinality of a generating family, under the regime declared in the source. Enlarging the acting algebra changes the question. Replacing algebraic span by a closure changes it again. The machine record therefore keeps the acting regime and hypotheses beside the statement.

Chapter I begins with typed operator grammars and their realizations, then passes through free presentations, generator number, packet coordinates, block algebras, transfer graphs, and realization. The resulting record graph covers the whole expanded foundation. It is not limited to the finite matrix examples that the runtime executes.

For finite families of nonnegative integers, the ambient convention is $\max \varnothing = 0$. The height of an empty finite poset is zero. A nonempty poset with no strict comparable pair also has height zero, so height counts strict steps. These conventions clarify the empty cases without adding stable records.

The source separates algebraic, linear, and topological claims. A finite-dimensional matrix computation can exercise a linear identity. It cannot establish the Banach space result in the countable setting or a generator formula over every division ring. Those claims retain their own source proofs and dependencies.

1.2 Exact packet coordinates

For a finite index set I, a packet system on X consists of linear maps P_α satisfying

$$P_\alpha^2 = P_\alpha, \qquad P_\beta P_\alpha = 0 \quad (\beta \neq \alpha), \qquad \sum_{\alpha \in I} P_\alpha = I_X.$$

The packet spaces are the ranges $X_\alpha = P_\alpha X$. Analysis and synthesis are

$$\mathcal{A}_P x = (P_\alpha x)_{\alpha \in I}, \qquad \mathcal{S}_P((x_\alpha)_\alpha) = \sum_\alpha x_\alpha.$$

Their inverse laws are exact. No assumption of inner product orthogonality is needed. This distinction is visible in the oblique two-dimensional example later in this companion.

Machine execution represents each packet as an ambient vector. It does not construct a basis of every range or replace the ambient representation by smaller coordinate arrays. This choice makes exact identities easy to compare with the source formulas while retaining the packet index structure.

1.3 Transfer blocks and composition

For an endomorphism T, the transfer block from packet α to packet β is

$$T_{\beta\alpha} = P_\beta T P_\alpha.$$

Its source is X_α and its target is X_β. In the finite runtime it is stored as a full ambient matrix. Completeness gives

$$T = \sum_{\alpha,\beta} T_{\beta\alpha}, \qquad (ST)_{\gamma\alpha} = \sum_{\beta} S_{\gamma\beta} T_{\beta\alpha}.$$

The order matters: the rightmost factor acts first. The runtime uses the same convention. Its nested block array is indexed first by target and then by source.

The graph records source dependencies for these identities. It does not infer every implicit mathematical dependency. The packet-system hypotheses remain required even when a compact source statement inherits them from the surrounding section.

1.4 Intertwining and reconstruction

For packet systems P on X and Q on Y, an intertwiner $U : X \to Y$ satisfies $UT = SU$. Resolving both sides yields

$$\sum_{\beta} U_{\gamma\beta} T_{\beta\alpha} = \sum_{\delta} S_{\gamma\delta} U_{\delta\alpha}, \qquad U_{\gamma\alpha} = Q_\gamma U P_\alpha.$$

All local equations together are equivalent to the global equation under the packet hypotheses. The runtime returns the global Boolean, every local residual, and the conjunction of the local Booleans. It provides the data needed to inspect agreement or failure in a concrete exact case.

1.5 The boundary of finite execution

Countable packet sums require a topology and a convergence condition. Quotient descent requires invariance. Coordinate transport requires a suitable invertible map. Packet-pure seed counts differ from unrestricted seed counts because one mixed-support seed can contribute to several source components. These are mathematical hypotheses, not formatting conventions.

The source records seven explicit boundaries. Their presence is part of the interface. Runtime action names do not imply that every associated theorem has an implementation, and metadata listing a boundary does not itself execute that boundary check.

2 Canonical machine representation

2.1 Entrypoint and package identity

The mathematical records are located at `formal/coverage/COVERAGE_108.json`. Appendix A is generated directly from those records. Each includes the statement, source proof where present, source location, hashes, formal status, and declaration correspondence. The archived mathematical source identified by those hashes is the statement authority.

The source register identifies the executable runtime. Runtime commands assume its working directory. Its entrypoint, PSC_I.machine.json, points to the preserved record collection, statement AST, dependency graph, action registry, and evidence index. A

runtime result therefore retains the exact interface scope explained below. It is not a certificate for every current manuscript statement.

The preserved runtime source digest is:

```
37bf8f03dbbf66ce41f8b07a8e7be8fa5fca44176bdd56dd25b483f72736ff77
```

That digest identifies the archived runtime source, including explicit nonzero-packet assumptions for two corollaries and the empty-family and poset-height conventions. The mathematical inventory has its own statement and proof hashes, printed in Appendix A. Runtime receipts remain attached to their original bytes. They do not transfer automatically to the mathematical inventory.

2.2 Record identity and source provenance

Each current record has a stable ID, kind, title, statement text, optional proof text, source locator, hashes, formal status, declaration targets, and a scope note. These fields identify the actual current statement and the proof text that accompanies it. The preserved executable payload additionally supplies its own dependency annotations, semantic profiles, evidence coordinates, and fingerprints for its source.

There are 28 definitions, 28 theorems, 22 corollaries, 16 propositions, three lemmas, three classical inputs, seven boundaries, and one remark. The 69 theorem/proposition/corollary/lemma records have attached proof bodies. Definitions and boundary statements are not counted as proved theorems merely because they occupy records.

> A fingerprint in the preserved runtime provides a content identity under that payload's generating convention. It is not a proof certificate. The runtime verifier checks the source digest, ID count, graph, AST coverage, selected API behavior, and exact fixtures. It does not establish all mathematical content through independent formal reconstruction.

2.3 Statement AST and typed signatures

The runtime statement AST has one entry per stable ID in its source. The statuses are 20 TYPED_CORE, 81 STRUCTURED_SYMBOLIC, and seven TEXTUAL_SEMANTIC. A typed-core record points to a signature declaring parameters, requirements where supplied, and a return description.

The AST segments statements into prose and mathematical surfaces. Some expressions are recognized as relations or direct-sum expressions. Others retain `latex_expr` leaves. Even a typed-core record can contain these leaves. Consequently, "typed core" refers to the attached interface signature, not to a complete formal elaboration or a proof term.

The signature for block intertwining describes a logical equivalence between the global and local equations. Its source proof remains attached separately. The runtime can evaluate a finite instance. These are three different representations with three different evidentiary roles.

2.4 Dependencies and closure

The preserved runtime dependency graph contains 108 nodes and 97 internal edges. The runtime verifier confirms that the recorded internal dependencies resolve and that the graph is acyclic. Direct dependencies are stored on each record. The API can also return a transitive closure.

Closure queries return sorted identifiers. They do not produce a proof order, carry out the proof, or resolve external classical inputs. A consumer can use the graph to inspect a prerequisite set and then obtain source statements and evidence for each record.

2.5 Operation profiles and actions

Each preserved runtime record carries source-derived operation metadata, with a transitive profile alongside the direct profile. The ontology distinguishes structural operations from hypothesis classes. These annotations support planning and inspection. They should not be interpreted as a verified extraction of every semantic operation used by an arbitrary proof.

The action registry contains 37 action names associated with 75 records. Eight names are implemented in the runtime. Other entries are marked planner-only or fixture-only. The source has broader mathematical scope than the executable subset.

3 Evidence semantics and certification scope

3.1 Independent evidence coordinates

The preserved runtime evidence object separates source, KIR, Python, and Lean. Its source coordinate identifies that payload's source, and its KIR coordinate records validation of that source. Python evidence reports exact finite fixtures for 26 records. Its Lean coordinate points to eight runtime source mappings.

The catalogue uses the formal correspondence map instead. Each of the 108 records carries a status and scope note, and each proof-level statement has its own correspondence assessment. The verification index distinguishes the execution evidence attached to each component.

No coordinate upgrades another automatically. Exact arithmetic removes floating-point ambiguity from a finite computation. It does not quantify over arbitrary fields, modules, dimensions, or topological spaces. A mapped Lean declaration proves its own formal statement under its dependencies. Its relation to the source statement must still be read from the mapping.

3.2 Formal correspondence and checks

The correspondence map classifies the 69 proof-level records as 69 covered, 0 partial, and 0 not mapped. The classifications describe the complete statements at their recorded scopes. They do not count independent theorems or executable actions. Every statement, source proof, and correspondence scope is reproduced in Appendix A. The exact declaration lists are in `formal/coverage/COVERAGE_108.json`.

The recorded audit checks 116 formal source modules plus the retained boundary suite. Including that suite, it inventories 3513 source-owned entries and checks 3500 logical declarations. Of these, 3499 are checked by logical replay and one declaration sharing an upstream name receives a fresh-name kernel body check after exact type/value comparison. It inventories 13 runtime helpers separately and records 12 rejected negative controls. The allowed transitive axiom set is `Classical.choice`, `Quot.sound`, `propext`. The receipt is `formal/evidence/RELEASE_VERIFICATION.json`. Its detailed compiler and logical-replay evidence is under `formal/evidence/whole_closure/`.

The recorded computational checks pass for 96 exact finite or symbolic visual assertions, 33 companion tests, and 10 regenerated PDF/PNG figure pairs. These bounded computations have their own scope and do not replace the formal declarations.

The pinned upstream dependency cache is reused. Logical replay uses Lean's official kernel. It is not an independently implemented checker. The source-to-manuscript correspondence is a separate reviewed mathematical artifact.

3.3 Runtime Lean linkage

The attached seven-module formalization uses the environment specified in the accompanying source records. Its eight mappings include definitions, generalized theorems, one-directional correspondence, and a related sufficient direction. The support-pattern algebra mapping is explicitly partial. It must not be counted as a full formal certificate for every direction of the source criterion.

A separate predecessor report records 35 of 35 obligations for its identified source. That lineage is not the canonical correspondence for the expanded mathematical inventory. The predecessor count, the eight attached runtime mappings, and the expanded correspondence map describe different scopes.

3.4 Recorded runtime checks

The documented verifier and exact fixture suite were run from the archived runtime payload. Both passed. The verification log is attached in the companion's `build/` directory. All 65 files extracted from the original machine archive retain their original bytes.

An initial invocation used an interpreter without SymPy and failed when the blockification action imported that dependency. The successful replay used an existing environment containing SymPy. No runtime action implementation was changed to resolve that environment failure. The two audited statement corrections were applied separately and have their own provenance record.

> Typesetting this companion does not execute Lean. Lean checks are reported in the verification index and explained in the formal companion. The runtime status file is retained with its original source identity.

3.5 What PASS means

A runtime-verifier pass means the named package checks succeeded. A fixture pass means a concrete exact computation satisfied its assertion. A transport-gate pass means all operation flags that this runtime checked were accepted. None of these strings, taken alone, means that the entire 108-record foundation has been formally proved by the runtime.

The full source proof catalog is included so that a reader can inspect mathematical arguments beyond the runtime. Editorial reproduction of those arguments is not an adversarial proof audit or a new formal verification claim.

4 Executable action contracts

4.1 Input and output conventions

The API is a Python command that emits JSON on successful calls. Exact arithmetic actions use SymPy matrices. Integers and rational strings such as `"3/7"` preserve rational values. JSON floating-point numbers should be avoided when the intended claim is an exact rational identity. Returned scalar entries are strings.

Payload shapes are conventional rather than comprehensively validated at the API boundary. Supply nonempty compatible square projector matrices and compatible operators or vectors. Malformed inputs can raise Python exceptions. The command parser does not present a universal contract for structured errors.

Most computational actions do not call packet validation automatically. Validate the packet family before interpreting block or analysis output through the theorems. With invalid projectors, a matrix expression may still be computable while its theorem interpretation fails.

4.2 The six exact finite actions

Action	Payload	Returned information
`VALIDATE_PACKET_SYSTEM`	`projectors`	Idempotence, annihilation, completeness, and their conjunction.
`ANALYZE_PACKETS`	`projectors`, `vector`	Ambient packet vectors and a reconstruction Boolean.
`BLOCKIFY_OPERATOR`	`projectors`, `operator`	Target/source array of ambient transfer matrices.
`RECONSTRUCT_OPERATOR`	`blocks`	Sum of every supplied block. Compatibility with a packet family is not independently checked.
`COMPOSE_TRANSFER_BLOCKS`	`projectors`, `S`, `T`	Array of block products and equality with directly resolved ST.
`LOWER_INTERTWINER`	`P`, `Q`, `T`, `S`, `U`	Global equation, local equations, residuals, and local conjunction.

4.3 The two semantic gate actions

`CHECK_REALIZATION_PROFILE` and `TRANSPORT_THEOREM` share the same runtime gate. Their payload contains a record ID and a profile. The gate reads that record's direct `action_required_operations` and checks the corresponding profile flags.

The accepted flags are true, `VERIFIED`, or `PRESERVED`. False and `NOT_PRESERVED` produce a negative list. Other or missing values are unknown. Any negative or unknown required operation blocks the gate.

This archived implementation does not evaluate the profile's `verified_hypotheses`, does not use the transitive operation closure as its required set, and does not execute the returned boundary guards. It also does not construct a target theorem, a proof object, or an independently verified realization functor. The preservation flags are declarations supplied by the caller.

Thus the action named `TRANSPORT_THEOREM` is an operation-profile gate. A consumer must separately justify target hypotheses, source-to-target interpretation, and any constructions used by the proof. Unknown flags fail closed only within the operation list actually inspected by the function.

4.4 Planner-only and fixture-only actions

The full registry in Appendix C makes execution status explicit. A planner-only action can identify a mathematically meaningful next step without implementing it. A fixture-only association records that a finite check exercises related mathematics. Calling an unimplemented action through `execute` exits with an unsupported-action error.

5 An exact worked execution

5.1 Oblique packets and reconstruction

Use

$$P_0 = \begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}, \qquad P_1 = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}, \qquad x = \begin{pmatrix} 3 \\ 5 \end{pmatrix}.$$

These are complementary idempotents but are not self-adjoint under the usual Euclidean inner product. The example therefore exercises the algebraic packet hypotheses without adding orthogonality.

```
python runtime/machine_api.py execute VALIDATE_PACKET_SYSTEM tests/packet_input.json
python runtime/machine_api.py execute ANALYZE_PACKETS tests/analyze_input.json
```


```
{
  "complete": true,
  "idempotent": true,
  "pairwise_annihilating": true,
  "valid": true
}
```


The analysis output is exact:


```
{
  "packets": [
    [
      [
        "-2"
      ],
      [
        "0"
      ]
    ],
    [
      [
        "5"
      ],
      [
        "5"
      ]
    ]
  ],
  "reconstructs": true
}
```


The two vectors $(-2, 0)^t$ and $(5, 5)^t$ sum to x. Each belongs to its own projector range. The runtime's zero-based packet indexing corresponds to the displayed P_0, P_1.

5.2 Resolving and rebuilding an operator

Take

$$T = \begin{pmatrix} 2 & 3 \\ 5 & 7 \end{pmatrix}.$$

The command `BLOCKIFY_OPERATOR` returns four ambient 2×2 matrices. In target/source order, they are

$$T_{00} = \begin{pmatrix} -3 & 3 \\ 0 & 0 \end{pmatrix}, \quad T_{01} = \begin{pmatrix} 0 & -7 \\ 0 & 0 \end{pmatrix}, \quad T_{10} = \begin{pmatrix} 5 & -5 \\ 5 & -5 \end{pmatrix}, \quad T_{11} = \begin{pmatrix} 0 & 12 \\ 0 & 12 \end{pmatrix}.$$

Their sum is T. The exact reconstruction API returns:


```
{
  "operator": [
    [
      "2",
      "3"
    ],
    [
      "5",
      "7"
    ]
  ]
}
```


The input and output JSON files for this reconstruction are supplied in `examples/`. The reconstruction action performs the block sum. Its interpretation as recovery of the original operator uses the validated packet system and the way those blocks were formed.

5.3 Composing resolved operators

Now set

$$S = \begin{pmatrix} 11 & 13 \\ 17 & 19 \end{pmatrix}, \qquad ST = \begin{pmatrix} 87 & 124 \\ 129 & 184 \end{pmatrix}.$$

The runtime forms the blocks of S and T, then sums over the middle packet index. It compares the resulting array with the directly resolved product. The supplied worked output has `equals_direct: true`. The full block arrays are retained in `examples/composition.json`.

The comparison checks a concrete instance of the multiplication identity. The general identity follows from inserting the complete family between the factors, as in the source proof reproduced in Appendix A.

5.4 An intertwining computation

Let

$$U = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad Q_i = UP_iU^{-1}, \qquad S' = UTU^{-1} = \begin{pmatrix} 7 & 3 \\ 5 & 2 \end{pmatrix}.$$

The target packets are $Q_0 = \left(\begin{smallmatrix} 1 & -2 \\ 0 & 0 \end{smallmatrix}\right)$ and $Q_1 = \left(\begin{smallmatrix} 0 & 2 \\ 0 & 1 \end{smallmatrix}\right)$. The global identity is $UT = S'U$.

The recorded execution of `LOWER_INTERTWINER` returns true for both the global identity and the conjunction of the four local equations. Each local residual is the zero matrix. One complete local result is:


```
{
  "holds": true,
  "lhs": [
    [
      "-3",
      "3"
    ],
    [
      "0",
      "0"
    ]
  ],
  "residual": [
    [
```

```
      "0",
      "0"
    ],
    [
      "0",
      "0"
    ]
  ],
  "rhs": [
    [
      "-3",
      "3"
    ],
    [
      "0",
      "0"
    ]
  ],
  "source": 0,
  "target": 0
}
```

This example deliberately changes packet coordinates as well as the operator. The complete input and four outputs are retained in `examples/intertwining_input.json` and `examples/intertwining.json`.

5.5 Inspecting the dependency set

The API exposes the dependencies used by the intertwining record linked to the source:

```
python runtime/machine_api.py deps PSC-I-T-BLOCK-INTERTWINING --transitive
```

```
{
  "dependencies": [
    "PSC-I-D-ANALYSIS-SYNTHESIS",
    "PSC-I-D-PACKET-SYSTEM",
    "PSC-I-D-TRANSFER-BLOCK",
    "PSC-I-T-BLOCK-MULTIPLICATION",
    "PSC-I-T-EXACT-PACKET-COORDINATES",
    "PSC-I-T-OPERATOR-RECONSTRUCTION"
  ],
  "record": "PSC-I-T-BLOCK-INTERTWINING"
}
```

This is a dependency closure, not a newly generated proof. Reading each named record supplies its exact source statement, proof, and evidence coordinates.

6 Transport profiles in use

6.1 Finite packet multiplication

The packet-linear profile declares preservation of the direct operations required by packet multiplication. The command

```
python runtime/machine_api.py transport PSC-I-T-BLOCK-MULTIPLICATION profiles/packet_linear.json
```

returns:

```
{
```

```
  "boundary_guards": [],
  "not_preserved": [],
  "record": "PSC-I-T-BLOCK-MULTIPLICATION",
  "required_operations": [
    "composition",
    "finite_sum",
    "packet_idempotents",
    "transfer_blocks"
  ],
  "status": "PASS",
  "unknown": []
}
```


The pass applies to the listed operation flags. It does not construct the target packet system or independently check a concrete functor's preservation laws.

6.2 A countable result is blocked

Using the same profile for the countable unconditional packet theorem returns:


```
{
  "boundary_guards": [],
  "not_preserved": [
    "completion",
    "topology"
  ],
  "record": "PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS",
  "required_operations": [
    "completion",
    "packet_idempotents",
    "topology"
  ],
  "status": "BLOCKED",
  "unknown": []
}
```


The profile does not preserve the topological and completion operations required by this gate. A finite linear profile therefore cannot be used to pass this countable request.

The source theorem additionally requires a real or complex Banach space, unconditional reconstruction, and a uniform multiplier bound. These hypotheses remain visible in its statement. Supplying permissive operation flags would not prove those hypotheses. The richer source contract and the narrower executable gate must be kept distinct.

6.3 Changing a profile without changing evidence

A profile file is a declaration supplied to the runtime. Editing its flags changes the gate's output. It does not create mathematical preservation evidence. A downstream system that wants a transport certificate must retain the arguments or formal certificates justifying those flags and the target hypotheses.

To use the gate, identify the exact record, inspect its source and dependency context, establish a target realization, justify the relevant preservation properties and hypotheses, then use the gate as an explicit consistency check. Its output can be attached to the resulting evidence record as one component.

7 Reproducing the computational material

7.1 Documented payload checks

From the runtime directory identified in the source register, use an interpreter containing SymPy:

```
python runtime/verify_release.py
python runtime/exact_fixture_suite.py
python runtime/nonzero_packet_regression.py
```

The first script checks entrypoint identity, record IDs, graph acyclicity and resolution, AST coverage, consistency of the action registry, source hash, selected API queries and actions, structured transport response, and fixture replay. The second emits the detailed fixture results. The latter does not replace a theorem prover.

7.3 A source change is a new evidence task

If a source statement changes, its source and statement identity change. The KIR, AST, dependency annotations, operation profile, formal map, and tests must be reviewed for impact. A recorded pass cannot be assigned to a new statement solely because the stable ID stayed the same.

The catalogue is generated from the archived mathematical source and records each statement and proof hash. The archived runtime payloads retain their own evidence. The additional traditional models and visual appendix are outside the 108-record inventory. They are not silently counted as extra machine records.

7.4 Runtime interfaces and mathematical statements

The source archive retains the computational source records. This companion supplies the statement catalogue and the runtime's exact transport-gate specification. Planner metadata remains distinct from executed proof machinery.

A Mathematical record catalogue

This catalogue is generated from the archived mathematical inventory's 108 stable records. Every statement and attached proof comes from that inventory. The source locator and statement hash identify the exact archived wording. Cross-references are linked to stable records in this catalog.

The formal-status line reports the reviewed correspondence map, not the older machine payload's eight-entry Lean map. Scope notes are part of each correspondence: "partial" is not a formal certificate for the complete source statement. The full declaration lists and source hashes are preserved in `formal/coverage/COVERAGE_108.json`.

The executable runtime contracts and runtime maps remain separate evidence for their stated scopes. Reproducing a source proof in this catalog does not give that proof a Lean certificate.

A.1 Foundations of seed generation

1. Coefficient and module regimes

Stable ID. `PSC-I-D-COEFFICIENT-REGIME`

Kind. definition

Statement.

All modules are left modules unless stated otherwise. The algebraic regime consists of a unital ring A and a unital left module M over A. When linear dimensions, tensor products, quivers, or operator blocks are used, A is a unital algebra over $\mathbb{k}$, where $\mathbb{k}$ is a field, and M is a vector space over $\mathbb{k}$. The topological regime consists of a Hausdorff topological vector space over a complete valued field, together with a unital algebra of continuous linear operators. Every infinite sum, completion, quotient topology, and closure below is interpreted only in the regime explicitly declared in its statement.

Formal correspondence. documentary. Regime declarations are distributed through Lean typeclass hypotheses, not one encoded regime object.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 7–10.

Statement SHA-256. `3350ba8d143f60c74ec0f3c1ea24b7208cc0346c6f292b8590f68a9520efd7cb`

2. Canonical theorem record

Stable ID. `PSC-I-D-THEOREM-RECORD`

Kind. definition

Statement.

A canonical theorem record is the tuple

$$(\text{ID}, \text{status}, \text{hypotheses}, \text{statement}, \text{dependencies}, \text{proof}, \text{exports}, \text{boundary}).$$

A downstream paper cites the stable record ID. A domain specialization may add domain hypotheses, but it does not repeat a generic proof from this paper.

Formal correspondence. documentary. Publication metadata. JSON records are not Lean proof declarations.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 14–22.

Statement SHA-256. `cd1763a1091beb27f77276664bfaa37d65f73f2b14b41b36f4b9692284853761`

3. Symbolic scope

Stable ID. `PSC-I-B-SYMBOLIC-SCOPE`

Kind. boundary

Statement.

This edition proves the algebraic and topological statements printed below. It does not assert historical novelty, independent external certification, automatic discovery of useful packet systems, nonlinear obstruction theory, multilevel refinement, arithmetic congruences, quantum gravity, automorphy, or local or global Langlands reciprocity.

Formal correspondence. scope boundary. Scope exclusion, not an additional positive theorem.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 25–28.

Statement SHA-256. `de436ff66bb1b236ea21bf9d3cf19c10b67981488cc3ab97eadc5b05708a39ba`

4. Typed operator grammar

Stable ID. `PSC-I-D-TYPED-GRAMMAR`

Kind. definition

Statement.

Let $Q = (Q_0, Q_1, s, t)$ be a quiver with finite vertex set. A typed operator grammar over $\Bbbk$ is a pair

$$\mathfrak{G} = (Q, I),$$

where $I \lhd \Bbbk Q$ is an ideal on both sides generated by linear combinations over $\Bbbk$ of paths having common source and common target. Write

$$\Lambda_{\mathfrak{G}} := \Bbbk Q / I.$$

A realization of $\mathfrak{G}$ is a family $(X_v)_{v \in Q_0}$ and linear maps

$$X_a : X_{s(a)} \longrightarrow X_{t(a)} \qquad (a \in Q_1)$$

that annihilate every relation in I. A morphism $f : X \to Y$ of realizations is a family of linear maps $f_v : X_v \to Y_v$ satisfying

$$f_{t(a)} X_a = Y_a f_{s(a)} \qquad (a \in Q_1).$$

Formal correspondence. definition represented. Finite vertex set, arbitrary arrow set, source/target maps, independent vector spaces and typed arrow maps; relations are finite linear combinations of paths with fixed endpoints. The more general free-algebra relation format is specialized both to homogeneous typed path relations and to all representatives of an actual TwoSidedIdeal.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 37–57.

Statement SHA-256. `282b22656cc22122458c5c399315c5468190e48cc5578ef5258b9a5246a905c9`

5. Collapsed one-space realization

Stable ID. `PSC-I-D-COLLAPSED-REALIZATION`

Kind. definition

Statement.

For a typed realization X, define its realization on a single space by

$$|X| := \bigoplus_{v \in Q_0} X_v.$$

Every vertex idempotent acts as a coordinate projector on $|X|$, and every path acts as a block operator from its source space to its target space.

Formal correspondence. definition represented. Total space is the finite product, canonically the finite direct sum. Vertex operators are single-after-projection maps and arrows are single-after-arrow-after-projection. Induction on the genuinely source/target-indexed TypedPath proves the actual block-composition action, including the zero-length vertex path.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 60–67.

Statement SHA-256. `79fcb85b9e889dae8d8305bb688123c8a54c9d9a18a5b86d37765b76b25a82aa`

6. Typed realization and module equivalence

Stable ID. `PSC-I-T-TYPED-MODULE-EQUIVALENCE`

Kind. theorem

Statement.

The category of realizations of $\mathfrak{G}$ is equivalent to the category of unital left modules over $\Lambda_{\mathfrak{G}}$. Explicitly, a realization X determines the collapsed space $|X|$ of `PSC-I-D-COLLAPSED-REALIZATION` with the vertex idempotent e_v acting as the coordinate projector onto X_v and each arrow acting in the block between its declared source and target. Conversely, a unital module M over $\Lambda_{\mathfrak{G}}$ determines the realization

$$M_v = e_v M, \qquad M_a(x) = ax.$$

The two constructions are inverse up to canonical natural isomorphism.

Proof. Because Q_0 is finite, $1 = \sum_{v \in Q_0} e_v$ in $\Bbbk Q$, the e_v are pairwise orthogonal, and the same identities hold in $\Lambda_{\mathfrak{G}}$. Given a realization, define the action of a path by composition of its arrow maps and extend linearly. Every element of I acts as zero, so the action factors through $\Lambda_{\mathfrak{G}}$. The vertex idempotents give

$$|X| = \bigoplus_v e_v|X| = \bigoplus_v X_v.$$

Conversely, if M is unital, then

$$M = 1M = \sum_v e_v M.$$

Orthogonality makes the sum direct. For an arrow $a : v \to w$, the identities $a = e_w a e_v$ imply

$$a(e_v M) \subseteq e_w M.$$

The relations in I act as zero because M is a module over $\Lambda_{\mathfrak{G}}$. Applying both constructions returns the original vertex spaces and arrow maps, or the original module decomposition, through the evident identity maps. These identifications commute with morphisms, giving an equivalence of categories. □

Formal correspondence. covered. The grammar algebra is constructed by FreeAlgebra and RingQuot, with complete orthogonal vertex idempotents and exact source/target arrow corners. Collapse is full, faithful and essentially surjective, using actual vertex ranges and their analysis/synthesis equivalence. Action objects are then proved equivalent to ordinary ModuleCat for all unital left modules; the coefficient action is recovered through algebraMap. A separate quotient comparison uses the actual TwoSidedIdeal ring congruence and proves both inverse identities. No category equivalence, split decomposition, path action or module structure is supplied as an assumption.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 70–80; proof lines 84–98.

Statement SHA-256. `a7fb9fcf8a2ff9a5b6ca94fba965afd801c3acb93f1fe6e0323b9fd3fab8470b`

7. Realized operator algebra

Stable ID. `PSC-I-D-REALIZED-ALGEBRA`

Kind. definition

Statement.

For a realization X, the realized operator algebra is

$$\mathcal{A}_X := \mathrm{im}(\Lambda_{\mathfrak{G}} \longrightarrow \mathrm{End}_{\Bbbk}(|X|)).$$

It records the actual operators in the realization and may be a proper quotient of the abstract grammar algebra.

Formal correspondence. definition represented. The realized algebra is literally the range of the constructed algebra homomorphism into Module.End. It is not a supplied equivalent algebra or an abstract surrogate.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 114–121.

Statement SHA-256. `43b505da15a63ccf3534f295a12aaadd6cd3d7d4f7e2ab5dba25d3d5cf63d0bc`

8. Action factorization and hull invariance

Stable ID. `PSC-I-P-ACTION-FACTORIZATION`

Kind. proposition

Statement.

Let $S \subseteq |X|$. The generated submodule under the grammar algebra equals the generated submodule under the realized operator algebra:

$$\Lambda_{\mathfrak{G}} S = \mathcal{A}_X S.$$

Consequently, every statement about seed rank depends only on the realized action, not on redundant relations in a chosen presentation of the grammar.

Proof. The action map $\rho : \Lambda_{\mathfrak{G}} \twoheadrightarrow \mathcal{A}_X$ is surjective by definition and satisfies $\lambda s = \rho(\lambda)s$. Hence every grammar orbit vector lies in $\mathcal{A}_X S$. The reverse inclusion holds because every element of $\mathcal{A}_X$ is $\rho(\lambda)$ for some λ. □

Formal correspondence. covered. The two actual spans are compared by span induction and range-surjectivity. The cardinal seed ranks are equal via the identity semilinear bijection into the actual image algebra. Arbitrary seed sets and zero modules are included; there is no finite-generation assumption.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 124–131; proof lines 135–137.

Statement SHA-256. `472b9117cf7ed5ed76713d4b131141c2e9f00b9b6dd827bcb2515705d21e94f1`

9. Linear seed system

Stable ID. `PSC-I-D-SEED-SYSTEM`

Kind. definition

Statement.

A linear seed system is a tuple

$$\mathfrak{S} = (X, \mathcal{A}, S, \mathcal{P}, \tau),$$

where X is a unital left module over $\mathcal{A}$, $\mathcal{A}$ is fixed independently of the target element, $S = (s_1, \ldots, s_r)$ is a finite distinguished seed family, $\mathcal{P}$ is either empty or a declared packet system, and τ is either the algebraic convention or a specified Hausdorff topology in which the declared operator action is continuous.

Formal correspondence. documentary. Algebraic and continuous packet cases are represented by concrete operator algebras, packet objects and arbitrary seed sets, which include every finite distinguished family after passage to its range. The record is a regime/data definition, not an assertion that one bundled structure covers the optional packet-free and topological regimes. Optional fields are represented in separate structures.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 145–152.

Statement SHA-256. `f1b9660868ad56e506bd33c2ab474312509199727cc8c23a64d4334641974e26`

10. Generated hull

Stable ID. `PSC-I-D-GENERATED-HULL`

Kind. definition

Statement.

For $S \subseteq X$, define

$$\mathrm{Hull}_{\mathcal{A}}(S) := \sum_{s \in S} \mathcal{A}s.$$

In the topological regime define

$$\mathrm{Hull}_{\mathcal{A},\tau}(S) := \overline{\mathrm{Hull}_{\mathcal{A}}(S)}^{\,\tau}.$$

For a singleton $S = \{s\}$, write $\mathrm{Hull}_{\mathcal{A}}(s)$.

Formal correspondence. definition represented. The scalar span of algebra-orbit vectors is proved equal as a set to the module span. The general topological operator hull is an actual Submodule and its topologicalClosure is used in the topological development.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 155–166.

Statement SHA-256. `859743fdac21ebb8ba120148cbf03f5511974f6e3356b5f733a3beea316b55f5`

11. Natural presentation

Stable ID. `PSC-I-D-NATURAL-PRESENTATION`

Kind. definition

Statement.

A seed presentation is natural relative to a declared grammar when the primitive operators and relations are fixed independently of the target, the relation system is finite or effectively enumerable, the topology is intrinsic to the realization, no primitive operator directly stores the complete target, and the presentation exposes a reusable invariant, theorem, or computation not encoded by a primitive chosen for the target.

Formal correspondence. documentary. A methodological criterion rather than a decidable Lean predicate.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 171–174.

Statement SHA-256. `94104ee5b6710359c3fa14a5febde34de8455668f83dfc5f49cf3e5028770033`

12. Algebra generation and module generation are distinct

Stable ID. `PSC-I-P-ALGEBRA-MODULE-GENERATION`

Kind. proposition

Statement.

Every nonzero unital algebra A, regarded as a left module over itself, has

$$\operatorname{srank}_A(A) = 1,$$

with cyclic seed 1_A. This conclusion is independent of the number of algebra generators required to present A as an algebra.

Proof. For every $a \in A$, one has $a = a1_A$. Hence $A1_A = A$. The number of operations needed to generate A as an algebra is a different invariant and does not enter the module calculation. □

Formal correspondence. covered. The regular action is multiplication on A. The singleton unit is proved generating by surjectivity of scalar evaluation; the nontrivial regular module then has seedRank exactly one. No assertion about algebra presentations is used as a premise.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 177–184; proof lines 188–190.

Statement SHA-256. `9ba2f9c1f32694b86a7d9ac63ef11d2b3c46ee5c38307f3abcc3c12048cf91d0`

13. Target-dependent operators

Stable ID. PSC-I-B-TARGET-DEPENDENT-OPERATORS

Kind. boundary

Statement.

For a fixed seed s and target x, the existence of a specially defined operator A_x satisfying $A_x s = x$ does not establish a natural seed calculus unless A_x belongs to a grammar fixed independently of the target satisfying PSC-I-D-NATURAL-PRESENTATION.

Formal correspondence. scope boundary. Target-independence restriction remains documentary.

Source locator. editable/traditional/02_foundations.tex; statement lines 192–195.

Statement SHA-256. aece204c76d6b378ec937375eebd02a43d0a48dbed1f94054b4ed514ef573dd1

14. Seed evaluation map

Stable ID. PSC-I-D-SEED-EVALUATION

Kind. definition

Statement.

Let A be a unital ring, let M be a left module over A, and let $S = (s_1, \dots, s_r) \in M^r$. Define the linear map over A

$$\varepsilon_S : A^{\oplus r} \longrightarrow M, \qquad (a_1, \dots, a_r) \longmapsto \sum_{j=1}^{r} a_j s_j.$$

Formal correspondence. definition represented. multiSeedMap is the actual Finsupp linear combination map; finite families give the stated finite free module.

Source locator. editable/traditional/02_foundations.tex; statement lines 215–223.

Statement SHA-256. db3e882c18dfdf5770d2779811295f6d287e9ffa95dbfb79bec513f96cba185d

15. Finite free presentation

Stable ID. PSC-I-T-FREE-PRESENTATION

Kind. theorem

Statement.

For the map in PSC-I-D-SEED-EVALUATION,

$$\operatorname{im} \varepsilon_S = \operatorname{Hull}_A(S), \qquad N_S := \operatorname{Ker} \varepsilon_S \leq A^{\oplus r},$$

and the induced map is an isomorphism of left modules over A

$$\boxed{A^{\oplus r} / N_S \cong \operatorname{Hull}_A(S).}$$

Consequently, S generates M if and only if ε_S is surjective.

Proof. The image consists exactly of finite sums $\sum_j a_j s_j$, which is $\operatorname{Hull}_A(S)$. Because ε_S is linear over A, its kernel is a left submodule. The first isomorphism theorem gives

$$A^{\oplus r} / N_S \cong \operatorname{im} \varepsilon_S = \operatorname{Hull}_A(S).$$

The final equivalence is the equality $\operatorname{im} \varepsilon_S = M$. □

Formal correspondence. covered. Actual range/span equality, quotient equivalence and generation/-surjectivity equivalence are present.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 226–239; proof lines 244–250.

Statement SHA-256. `6e2db91e9df6b69a04e531d9b552fe7148f878663bcd3600c36c94ac62592ef8`

16. Minimal free-cover characterization

Stable ID. `PSC-I-C-MINIMAL-FREE-COVER`

Kind. corollary

Statement.

For every finitely generated left module M over A,

$$\boxed{\operatorname{srank}_A(M) = \min\{r \geq 0 : \text{there exists a surjection } A^{\oplus r} \twoheadrightarrow M\}.}$$

Proof. A seed family of size r gives a surjection by `PSC-I-T-FREE-PRESENTATION`. Conversely, the images of the standard basis vectors under any surjection $A^{\oplus r} \twoheadrightarrow M$ form a generating family of size at most r. □

Formal correspondence. covered. seedFinrank_isLeast_free_surjection proves the finite minimum, not an executable basis algorithm.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 267–275; proof lines 279–281.

Statement SHA-256. `8e4ae31565265679e9c8bf397291baf65b396ad21bb88e7f00b8d17328b1cb00`

17. One-seed presentation

Stable ID. `PSC-I-C-ONE-SEED-PRESENTATION`

Kind. corollary

Statement.

For $s \in M$, the annihilator

$$\operatorname{Ann}_A(s) := \{a \in A : as = 0\}$$

is a left ideal and

$$\boxed{A/\operatorname{Ann}_A(s) \cong \operatorname{Hull}_A(s).}$$

Proof. The evaluation map for one seed is $a \mapsto as$. Its kernel is $\operatorname{Ann}_A(s)$, which is therefore a left ideal, and `PSC-I-T-FREE-PRESENTATION` gives the quotient isomorphism. □

Formal correspondence. covered. oneSeedMap has the stated annihilator kernel and its quotient is the literal cyclic range.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 283–293; proof lines 297–299.

Statement SHA-256. `92a43462490ec1465fe4129738284f9c3f53e2d8bb7b63f980c1d03a0fbbb2ed`

18. Two-sided annihilator criterion

Stable ID. `PSC-I-P-ANNIHILATOR-TWO-SIDED`

Kind. proposition

Statement.

For $s \in M$, the left ideal $L = \mathrm{Ann}_A(s)$ is an ideal on both sides if and only if

$$L\,\mathrm{Hull}_A(s) = 0.$$

If s is cyclic, these conditions are equivalent to

$$\mathrm{Ann}_A(s) = \mathrm{Ann}_A(M).$$

Proof. The left ideal L is an ideal on both sides exactly when $ab \in L$ for every $a \in L$ and $b \in A$. This is equivalent to

$$0 = (ab)s = a(bs) \qquad (a \in L,\ b \in A).$$

By additivity, the last identity is equivalent to $L\,\mathrm{Hull}_A(s) = 0$.

If s is cyclic, then $\mathrm{Hull}_A(s) = M$. Hence the preceding condition is $L \subseteq \mathrm{Ann}_A(M)$. The reverse inclusion always holds because an operator annihilating all of M annihilates s. Thus equality follows. Conversely, $\mathrm{Ann}_A(M)$ is an ideal on both sides, so equality implies that L is an ideal on both sides. □

Formal correspondence. covered. vectorAnnihilator is the actual kernel of a↦a•s. Right multiplication closure is equivalent to annihilating Submodule.span A {s}; Ideal.IsTwoSided supplies the standard two-sided-ideal interface. Under singleton generation, the actual ideal equals Module.annihilator A M. Noncommutative order is a*b and (a*b)•s=a•(b•s).

Source locator. `editable/traditional/02_foundations.tex`; statement lines 302–312; proof lines 316–325.

Statement SHA-256. `dd991bd6ce28ee7f94302ed9dd3fb00719cf26de3b6ccf740f441b118bdd8e54`

19. Completion of a dense free presentation

Stable ID. `PSC-I-T-COMPLETED-PRESENTATION`

Kind. theorem

Statement.

Let X be a complete Hausdorff locally convex space, let $\mathcal{A} \subseteq \mathrm{End}(X)$ act continuously, and let $S = (s_1, \dots, s_r)$. Put

$$E := \mathcal{A}^{\oplus r} / N_S$$

algebraically and let $j : E \to X$ be the injective map induced by ε_S. Give E the locally convex topology transported from $j(E)$, equivalently the topology generated by

$$\widetilde{p}(e) := p(j(e))$$

for the continuous seminorms p on X. Every $a \in \mathcal{A}$ acts continuously on E in the transported topology and therefore extends to $\widehat{E}$. If $\mathrm{Hull}_{\mathcal{A}}(S)$ is dense in X, then j extends uniquely to a topological isomorphism of modules over $\mathcal{A}$

$$\boxed{\widehat{E} \cong X.}$$

Proof. By PSC-I-T-FREE-PRESENTATION, j is an algebraic isomorphism of modules over $\mathcal{A}$ from E onto $\mathrm{Hull}_{\mathcal{A}}(S)$. The transported seminorms make it a topological isomorphism onto that subspace. For $a \in \mathcal{A}$ and a continuous seminorm p on X, continuity of $a : X \to X$ supplies continuous seminorms controlling $p(aj(e))$, so the induced action on E is continuous and extends uniquely to $\widehat{E}$. The subspace is dense by hypothesis. A Cauchy net in E is mapped to a Cauchy net in X, which converges because X is complete. This defines a continuous extension $\widehat{j} : \widehat{E} \to X$. For any $x \in X$, choose a net (e_λ) in E with $j(e_\lambda) \to x$. The transported topology makes (e_λ) Cauchy, so it converges to some $\xi \in \widehat{E}$, and continuity gives $\widehat{j}(\xi) = x$. Thus $\widehat{j}$ is surjective. If $\widehat{j}(\xi) = 0$, represent ξ by a Cauchy net (e_λ). Then $j(e_\lambda) \to 0$, so every defining seminorm $\widetilde{p}(e_\lambda)$ tends to zero. Thus $e_\lambda \to 0$ in the completion and $\xi = 0$. Hence $\widehat{j}$ is bijective. Its inverse is continuous by the universal property of completion, or directly because the topology on E was transported from X. □

Formal correspondence. covered. The actual finite free evaluation A^(Fin r)→X is used, its actual kernel is quotiented, and UniformSpace.comap pulls back the target uniformity to that quotient. Dense range is derived from dense module span. AbstractCompletion.compareEquiv supplies a uniform equivalence, linearity over A is proved by completion density, and uniqueness follows from equality on the dense image. Constant action uniform continuity is inherited through the actual quotient embedding; scalar action on Completion is the usual extension. The local-convexity pullback lemma uses ordered scalars (in particular ℝ, including underlying real spaces for complex spaces); the stronger uniform-module completion proof does not require local convexity.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 333–347; proof lines 351–353.

Statement SHA-256. `23708a85485e7162e31b64ee0a8fb316bec231a1aadaa0e00f0a08f70dfe1721`

20. Seed rank

Stable ID. PSC-I-D-SEED-RANK

Kind. definition

Statement.

For a unital left module M over A, define

$$\mathrm{srank}_A(M) := \min\{|S| : AS = M\}.$$

For a topological module (X, τ), define

$$\mathrm{srank}_{\mathcal{A},\tau}(X) := \min\{|S| : \overline{\mathcal{A}S}^{\tau} = X\}.$$

Set $\mathrm{srank}_A(0) = 0$. When the value is finite, it is the minimal number of module generators.

Formal correspondence. definition represented. The operator hull is the scalar span of actual action values; for an acting algebra this is AS. The cardinal infimum ranges over all densely generating subsets, and existence of a minimum is proved from identity membership and cardinal well ordering.

Empty generators and infinite cardinals are included. The direct moduleSeedRank definition uses Dense of the literal acting-ring module span, and a proved equality identifies it with the integer-linear operator-family definition even for a nonfaithful action.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 370–381.

Statement SHA-256. `81a23b39cf2b6194ae8b0bc84ac270ee5ef86e4703b5fb5411f5fa9519aafae9`

21. Cyclic characterization

Stable ID. `PSC-I-P-CYCLIC-CHARACTERIZATION`

Kind. proposition

Statement.

For $M \neq 0$,

$$\operatorname{srank}_A(M) = 1 \quad \Longleftrightarrow \quad M \text{ contains a cyclic vector.}$$

Proof. Both conditions say that there is $m \in M$ with $Am = M$. □

Formal correspondence. covered. The nontriviality hypothesis agrees with the current publication.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 384–392; proof lines 396–398.

Statement SHA-256. `de62f3fddcf5cefb429536723a8933a119585faa47b6f2e51745ea055ae6a9cc`

22. Monotonicity under enlargement of the acting algebra

Stable ID. `PSC-I-P-ACTING-ALGEBRA-MONOTONICITY`

Kind. proposition

Statement.

Let $B \subseteq A \subseteq \operatorname{End}_{\mathbb{k}}(X)$ be unital operator algebras with the same identity. Then

$$\boxed{\operatorname{srank}_A(X) \leq \operatorname{srank}_B(X).}$$

The same statement holds for topological seed rank when both algebras act continuously.

Proof. For every $S \subseteq X$, one has $BS \subseteq AS$. Hence every generating family over B is also a generating family over A. Taking closures gives the topological statement. □

Formal correspondence. covered. For unital operator algebras, actual family inclusion implies operator-hull inclusion. Closure monotonicity transports a minimum generating subset. This exactly supplies the topological clause, combined with the previously mapped algebraic semilinear rank theorem.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 400–407; proof lines 411–413.

Statement SHA-256. `115d0049078122c1d6c43bb8ec53237a6efae32ef13330b8e31d8bf77e68eb56`

23. Quotient monotonicity

Stable ID. `PSC-I-T-QUOTIENT-MONOTONICITY`

Kind. theorem

Statement.

Let $\varphi : A \to B$ be a unital ring homomorphism and let

$$\Phi : M \twoheadrightarrow N$$

be a surjective additive map satisfying

$$\Phi(am) = \varphi(a)\Phi(m) \qquad (a \in A,\ m \in M).$$

Then

$$\boxed{\operatorname{srank}_B(N) \le \operatorname{srank}_A(M).}$$

If M and N are topological modules, Φ is continuous and surjective, and the actions are continuous, then

$$\operatorname{srank}_{B,\tau_N}(N) \le \operatorname{srank}_{A,\tau_M}(M).$$

Proof. Let S generate M. For every $n \in N$, choose $m \in M$ with $\Phi(m) = n$ and write $m = \sum_i a_i s_i$. Then

$$n = \sum_i \varphi(a_i)\Phi(s_i),$$

so $\Phi(S)$ generates N.

For the topological statement, let S generate M densely. Given $n = \Phi(m)$, choose a net $m_\lambda \in AS$ converging to m. Continuity gives

$$\Phi(m_\lambda) \longrightarrow n,$$

and every $\Phi(m_\lambda)$ lies in $B\Phi(S)$. Thus $B\Phi(S)$ is dense in N. □

Formal correspondence. covered. The new theorem starts with the printed arbitrary unital ring homomorphism, additive map, scalar compatibility, continuity, and surjectivity. Span induction maps the actual source module span into the target module span; continuous surjectivity transports density. An attained minimum generating set maps to an image of no larger cardinality. No ring-homomorphism surjectivity or faithfulness is assumed. The direct module-span rank is proved equal to the earlier operator-family rank using integer-linear acting operators. This supplies the exact topological clause alongside the separately accepted algebraic theorem.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 421–440; proof lines 445–457.

Statement SHA-256. `ccde1353d26e9f7bc7aef341c35be5304729daa69f8f42274ab70e2f9d7016a7`

24. Isomorphism invariance

Stable ID. `PSC-I-C-ISOMORPHISM-INVARIANCE`

Kind. corollary

Statement.

An intertwining module isomorphism preserves algebraic seed rank. A topological intertwining isomorphism preserves topological seed rank.

Proof. Apply `PSC-I-T-QUOTIENT-MONOTONICITY` to the isomorphism and its inverse. □

Formal correspondence. covered. A continuous semilinear module equivalence with continuous inverse preserves the direct module-span cardinal rank, by applying the exact semilinear surjection inequality in both directions. The same-ring case uses identity ring maps. The earlier bidirectional operator-family theorem covers intertwining operator-algebra isomorphisms, and the new rank equality bridges these definitions. This supplies the exact topological clause alongside separately accepted algebraic invariance.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 464–467; proof lines 471–473.

Statement SHA-256. `fc01d1773cdeebcaba8346b323df6df33d48e0f6f7adda763b14307b2e902ca1`

25. Short exact sequence bounds

Stable ID. `PSC-I-T-EXACT-SEQUENCE-BOUNDS`

Kind. theorem

Statement.

Let

$$0 \longrightarrow L \xrightarrow{\iota} M \xrightarrow{\pi} N \longrightarrow 0$$

be a short exact sequence of left modules over A. Then

$$\boxed{\operatorname{srank}_A(N) \leq \operatorname{srank}_A(M) \leq \operatorname{srank}_A(L) + \operatorname{srank}_A(N).}$$

Proof. The lower inequality is quotient monotonicity. For the upper inequality, choose generating families $(\ell_i)_{i\in I}$ of L and $(n_j)_{j\in J}$ of N with the least possible cardinalities. Choose a lift $m_j \in M$ of each n_j. For any $m \in M$, a finite subset $F \subseteq J$ and coefficients $a_j \in A$ give

$$\pi(m) = \sum_{j\in F} a_j n_j.$$

The vector $m - \sum_{j\in F} a_j m_j$ belongs to $\iota(L)$, so it is a finite linear combination of the $\iota(\ell_i)$. The union of the lifted family and the image of the generating family of L therefore generates M. Its cardinality is at most $|I| + |J|$, giving the upper bound. This argument uses finite sums for each vector even when either generating family is infinite. □

Formal correspondence. covered. Actual linear f,g with injectivity, range f=ker g and surjectivity; canonical seedRank=top.spanRank. Minimum generators of N are lifted and combined with images of

minimum generators of L. The proof derives generation of M and the cardinal upper bound; the lower bound is surjective monotonicity. No splitting, finite generation or desired bound is assumed.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 475–488; proof lines 492–498.

Statement SHA-256. `2e4e5af8593f49860709b8eb0b2d2b88577c0bc765a8b9e0c086441aaf142348`

26. Shared direct-sum bounds

Stable ID. `PSC-I-P-SHARED-DIRECT-SUM`

Kind. proposition

Statement.

If one algebra A acts diagonally on $M \oplus N$, then

$$\boxed{\max\{\operatorname{srank}_A(M), \operatorname{srank}_A(N)\} \leq \operatorname{srank}_A(M \oplus N) \leq \operatorname{srank}_A(M) + \operatorname{srank}_A(N).}$$

Proof. The coordinate projections are intertwining quotients and give the lower bound. For the upper bound, embed minimal generating families of M as $(m_i, 0)$ and of N as $(0, n_j)$. Their union generates by finite sums and has cardinality at most the sum of the two seed ranks. □

Formal correspondence. covered. The actual diagonal A-action on M×N realizes the binary direct sum. Projection surjectivity gives both lower bounds; inclusion of generating sets in the two factors yields cardinal addition for the upper bound.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 506–516; proof lines 520–522.

Statement SHA-256. `613d0bed5e9f4890c069d8d37670d0b8c9cb3a981df5bceb95686c08dc5638c6`

27. Independent direct-sum formula

Stable ID. `PSC-I-T-INDEPENDENT-DIRECT-SUM`

Kind. theorem

Statement.

Let M be a left module over A and N a left module over B. Under the action of the product algebra

$$(a, b)(m, n) = (am, bn),$$

one has

$$\boxed{\operatorname{srank}_{A\times B}(M \oplus N) = \max\{\operatorname{srank}_A(M), \operatorname{srank}_B(N)\}.}$$

Proof. The coordinate projections give the lower bound. Let r be the maximum on the right, regarded as a cardinal, and choose an index set I of cardinality r. Choose minimal generating families for M and N, index them by subsets of I, and fill any unused positions with zeros. Write the resulting families as $(m_i)_{i\in I}$ and $(n_i)_{i\in I}$. For any $m \in M$ and $n \in N$, their finite expressions involve a common finite subset $F \subseteq I$ after zero coefficients have been added. Thus

$$m = \sum_{i\in F} a_i m_i, \qquad n = \sum_{i\in F} b_i n_i,$$

and

$$(m, n) = \sum_{i \in F} (a_i, b_i)(m_i, n_i).$$

The family of paired vectors generates and has cardinality at most r. The empty family handles the case in which both modules are zero. □

Formal correspondence. covered. Actual product-algebra action and the maximum formula are proved.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 524–538; proof lines 542–554.

Statement SHA-256. `b6f497941ccc0be69e3780d70cd1d79cb39363dedb591b81527183724fc58748`

28. Tensor-product bound

Stable ID. `PSC-I-P-TENSOR-BOUND`

Kind. proposition

Statement.

Let A and B be unital algebras over $\mathbb{k}$, let M be a left module over A, and let N be a left module over B. Under the natural $A \otimes_{\mathbb{k}} B$ action,

$$\boxed{\operatorname{srank}_{A \otimes B}(M \otimes_{\mathbb{k}} N) \leq \operatorname{srank}_A(M) \operatorname{srank}_B(N).}$$

Proof. Choose minimal generating families $(m_i)_{i \in I}$ for M and $(n_j)_{j \in J}$ for N. Each vector in either module is a finite linear combination of its generators. Bilinearity therefore expresses every simple tensor as a finite sum of terms

$$(am_i) \otimes (bn_j) = (a \otimes b)(m_i \otimes n_j).$$

Simple tensors span the algebraic tensor product, so the family $(m_i \otimes n_j)_{(i,j) \in I \times J}$ generates it. This family has cardinality at most $|I||J|$, which is the claimed bound. The proof also covers an empty generating family, when the corresponding module and tensor product are zero. □

Formal correspondence. covered. The genuine tensor-product action and cardinal product upper bound are proved.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 557–567; proof lines 571–577.

Statement SHA-256. `b1b31d79b3ffc3d73319922c4a3f2f1dde12a0f774ceda1ae29227e9c63a6d4e`

29. Tensor rank is not generally multiplicative

Stable ID. `PSC-I-B-TENSOR-NONEQUALITY`

Kind. boundary

Statement.

The inequality in `PSC-I-P-TENSOR-BOUND` can be strict. Equality requires additional hypotheses on the acting algebras and their module multiplicities and is not exported as a universal law.

Formal correspondence. scope boundary. No universal tensor equality is exported. This boundary is not counted as a positive theorem.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 584–587.

Statement SHA-256. `9328501ddd219b9e0defabc4233a3e0c6437a8890f6920385de876bc5223e204`

30. Scalar-extension monotonicity

Stable ID. `PSC-I-P-SCALAR-EXTENSION-MONOTONICITY`

Kind. proposition

Statement.

Let $K/\Bbbk$ be a field extension, let A be a unital algebra over $\Bbbk$, and let M be a finitely generated left module over A. Set

$$A_K := K \otimes_{\Bbbk} A, \qquad M_K := K \otimes_{\Bbbk} M.$$

Then

$$\boxed{\operatorname{srank}_{A_K}(M_K) \leq \operatorname{srank}_A(M).}$$

Every generating family $S \subseteq M$ extends to the generating family $1 \otimes S \subseteq M_K$.

Proof. Let $S = \{s_1, \ldots, s_r\}$ generate M. Every pure tensor $\lambda \otimes m \in M_K$ has an expression

$$m = \sum_{i=1}^{r} a_i s_i,$$

so

$$\lambda \otimes m = \sum_{i=1}^{r} (\lambda \otimes a_i)(1 \otimes s_i).$$

Pure tensors span M_K, hence $1 \otimes S$ generates M_K over A_K. Taking the minimum over S proves the inequality. □

Formal correspondence. covered. The source constructs the natural (K⊗k A)-action on K⊗k M using commuting tensor-factor actions and proves (c⊗a)•(d⊗m)=(c*d)⊗(a•m). Tensor and span induction prove the literal image (1⊗-) of any generating set generates. Minimum-cardinality generation plus cardinality of an image gives the rank inequality.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 592–607; proof lines 612–624.

Statement SHA-256. `89013f407adf17424cd55db8a6f59a4a9141e34d39a1d350731d9e18d542a3e4`

31. Seed-rank witness

Stable ID. `PSC-I-D-SEED-RANK-WITNESS`

Kind. definition

Statement.

A surjective intertwiner $\Phi : M \twoheadrightarrow N$ is a witness for seed rank of order r when $\operatorname{srank}(N) = r$. It certifies $\operatorname{srank}(M) \geq r$.

Formal correspondence. documentary. A named use of the quotient lower bound, not a new proof target.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 632–635.

Statement SHA-256. `81381ceea3d420ff6ba2f0b9378863598308dc392cf597cc1dc0b4d7f84f7cc0`

32. Scalar quotient witness

Stable ID. `PSC-I-P-SCALAR-QUOTIENT-WITNESS`

Kind. proposition

Statement.

Let M be a module over A with coefficient field $\Bbbk$. Suppose M has a quotient N of dimension r on which A acts through a character

$$\chi : A \longrightarrow \Bbbk.$$

Then

$$\boxed{\operatorname{srank}_A(M) \geq r.}$$

Proof. For every $n \in N$, the cyclic submodule An is contained in the scalar span $\Bbbk n$. Therefore q seed images span a subspace of dimension at most q. Generating the quotient of dimension r requires at least r seeds. Quotient monotonicity transfers the lower bound to M. □

Formal correspondence. covered. The character, surjective semilinear map and finite scalar dimension match the printed hypotheses.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 638–648; proof lines 652–654.

Statement SHA-256. `92e35ac7e226b34fce4aa0e1e40a198c1273cc439ac42a8ea2f437f75965a632`

33. Nakayama lemma

Stable ID. `PSC-I-CI-NAKAYAMA`

Kind. classicalinput

Statement.

Let A be a unital ring, let $J \subseteq \mathrm{Jac}(A)$, and let M be a finitely generated left module over A. If $M = N + JM$ for a submodule $N \leq M$, then $M = N$.

Formal correspondence. classical input integrated. The noncommutative input is integrated at the printed finitely generated left-module scope. For any left ideal I contained in Ring.jacobson R, the actual submodule equality N ⊔ I • top = top implies N = top. No commutativity, Noetherian, semisimplicity, finite length, or two-sidedness of I is required for this lifting theorem. The proof derives the radical lifting result from maximal submodules and the ring-radical action inclusion, rather than assuming the desired conclusion.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 669–672.

Statement SHA-256. `68901a557c34762568711f0e6c8b400485a0c4b66477b8f231e899d9a48f5878`

34. Generator number is detected modulo the Jacobson radical

Stable ID. `PSC-I-T-MODULE-TOP`

Kind. theorem

Statement.

Let A be a unital ring, let $J = \mathrm{Jac}(A)$, and let M be a finitely generated left module over A. A family $S \subseteq M$ generates M if and only if its image generates M/JM as a module over A/J. When A/J is semisimple Artinian, this quotient is the usual semisimple top. No semisimplicity of A/J is required for the general equality. Consequently,

$$\boxed{\mathrm{srank}_A(M) = \mathrm{srank}_{A/J}(M/JM).}$$

Proof. If S generates M, its image generates every quotient. Conversely, suppose the image of S generates M/JM and put $N = AS$. Then

$$M = N + JM.$$

Nakayama's lemma gives $M = N$. Thus the same finite families generate before and after passage to the top, and the minimal cardinalities agree. □

Formal correspondence. covered. The complete generating-set/family equivalence and canonical cardinal seedRank equality are proved for arbitrary unital rings and finitely generated left modules using the literal M / (Jac(R) • top), its actual scalar action over R / Jac(R), and an exact quotient-map kernel. Any lifts of a generating quotient family generate upstairs, and both minimum-cardinality inequalities are established. The natural seedFinrank equality is proved as well. No semisimplicity hypothesis is needed. The descriptive semisimple-top sentence is covered by the pinned general IsSemisimpleRing.isSemisimpleModule instance applied to this actual quotient module. This is not an algorithm for computing the radical or finding generators.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 675–683; proof lines 688–694.

Statement SHA-256. `8e3170bbaf10331d7d774260fbdf4b7df0ca59f1c23116f008a1aa5ac2bf07a8`

35. Local algebra formula

Stable ID. `PSC-I-C-LOCAL-ALGEBRA-FORMULA`

Kind. corollary

Statement.

Let A be a local ring with Jacobson radical J and residue division ring $D = A/J$. For every finitely generated left module M over A,

$$\boxed{\operatorname{srank}_A(M) = \dim_D(M/JM),}$$

where the dimension is that of a left vector space over D.

Proof. The module M/JM over A/J is a left vector space over D. Its minimal number of generators is its dimension over D. □

Formal correspondence. covered. The local formula is proved for noncommutative local rings and finitely generated left modules. From IsLocalRing R the source proves uniqueness of the maximal left ideal, identifies the actual Jacobson radical as maximal, constructs DivisionRing (R / Jac(R)) internally, and proves the canonical seedRank equals the left vector-space Module.rank of the actual radical quotient. The natural finite-rank equality is also proved. No commutative residue-field surrogate or caller-supplied division-ring certificate is used.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 709–718; proof lines 722–724.

Statement SHA-256. `50bd885a16ef40862a42e9ddd15b7dd03758b5ee514ee98aab28acf22c1e26ed`

36. Wedderburn–Artin decomposition

Stable ID. `PSC-I-CI-WEDDERBURN-ARTIN`

Kind. classicalinput

Statement.

Every semisimple Artinian ring is isomorphic to a finite product

$$A \cong \prod_{i=1}^{q} M_{d_i}(D_i)$$

of full matrix rings over division rings.

Formal correspondence. classical input integrated. The actual arbitrary-division-ring Wedderburn-Artin existence theorem is now integrated at its semisimple-ring scope. The finite-algebra radical quotient is proved semisimple and its actual decomposition is constructed. These depend on proved pinned Mathlib results, not a custom existence axiom. Compatible finite module decompositions are separately constructed in ProductPresentation and SemisimpleExistence.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 727–734.

Statement SHA-256. `effdfe288b991fcc5cdf7549800403bbb6fd698be57f017880428042cc23a058`

37. Regular-module multiplicities

Stable ID. `PSC-I-L-REGULAR-MODULE-MULTIPLICITIES`

Kind. lemma

Statement.

Let

$$A = \prod_{i=1}^{q} M_{d_i}(D_i)$$

and let $S_i = D_i^{d_i}$ denote the natural simple left module for the ith factor, extended by zero across the other factors. Then, as a left module over A,

$$\boxed{A \cong \bigoplus_{i=1}^{q} S_i^{\oplus d_i}.}$$

Proof. For one factor $M_d(D)$, let e_{jj} be the diagonal matrix units. The left regular module decomposes as

$$M_d(D) = \bigoplus_{j=1}^{d} M_d(D) e_{jj}.$$

Each minimal left ideal $M_d(D)e_{jj}$ is isomorphic to the natural column module D^d. Taking the direct product of factors gives the stated decomposition. □

Formal correspondence. covered. The actual left regular module is explicitly equivalent to its natural columns, then to the displayed finite direct sum of natural-module copies over arbitrary division rings. Coordinate ring homomorphisms define the inflated actions, other factors act by zero, and positive-dimensional natural factors are proved simple. No transpose-as-ring-map or commutative-scalar shortcut is used. Finite products for the inner repeated copies are the canonical finite direct-sum model.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 737–747; proof lines 751–757.

Statement SHA-256. `cd1bf9aa7ec7ad71f24491e1969c318f330a268fb73a29873ff2e9494eb91799`

38. Semisimple seed-rank formula over arbitrary division rings

Stable ID. `PSC-I-T-SEMISIMPLE-SEED-RANK`

Kind. theorem

Statement.

Let

$$A \cong \prod_{i=1}^{q} M_{d_i}(D_i)$$

be semisimple Artinian and let

$$M \cong \bigoplus_{i=1}^{q} S_i^{\oplus m_i}$$

be a finitely generated left module over A. Then

$$\boxed{\operatorname{srank}_A(M) = \max_{1 \le i \le q} \left\lceil \frac{m_i}{d_i} \right\rceil .}$$

If A is any algebra of finite dimension over a field, the same formula applied to the decomposition of $M/\operatorname{Jac}(A)M$ over $A/\operatorname{Jac}(A)$ computes $\operatorname{srank}_A(M)$.

Proof. By `PSC-I-C-MINIMAL-FREE-COVER`, M is generated by r elements exactly when it is a quotient of $A^{\oplus r}$. By `PSC-I-L-REGULAR-MODULE-MULTIPLICITIES`,

$$A^{\oplus r} \cong \bigoplus_{i} S_i^{\oplus r d_i}.$$

Every short exact sequence of modules over a semisimple ring splits. Therefore a quotient isomorphic to M exists if and only if

$$m_i \le r d_i \qquad \text{for every } i.$$

The smallest such r is the displayed maximum. For a nonsemisimple algebra of finite dimension, `PSC-I-T-MODULE-TOP` reduces the calculation to the semisimple module $M/\operatorname{Jac}(A)M$ over $A/\operatorname{Jac}(A)$. □

Formal correspondence. covered. Actual product matrix ring with left multiplication and natural column modules acted on through the matching factor. Semilinear eM along eR enforces compatibility. DivisionRows proves left generation iff row spanning and m<=r*d; standard rows prove sufficiency. Product assembly, column rearrangement and seed-rank invariance prove the formula. RadicalTop is literally M/(Jac(R) smul top); existence constructs decompositions.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 759–779; proof lines 784–795.

Statement SHA-256. `c604c6473ceeb6b651f37301bd3e719d69a555f04c9e4aea5fa6ad49bcd35de4`

39. Algebraically closed multiplicity formula

Stable ID. `PSC-I-C-ALGEBRAICALLY-CLOSED-FORMULA`

Kind. corollary

Statement.

Let $\Bbbk$ be algebraically closed, let A be a unital algebra of finite dimension over $\Bbbk$, put $J = \mathrm{Jac}(A)$, and let M be a finitely generated left module over A. Assume compatible decompositions indexed by an integer $q \geq 0$:

$$A/J \cong \prod_{i=1}^{q} M_{d_i}(\Bbbk), \qquad M/JM \cong \bigoplus_{i=1}^{q} (\Bbbk^{d_i})^{\oplus m_i},$$

where the d_i are positive integers and the m_i are nonnegative integers. The first isomorphism is an isomorphism of algebras over $\Bbbk$. The second is an isomorphism of left modules over A/J, using the natural column module for each matrix factor and zero action from the other factors. Then

$$\mathrm{srank}_A(M) = \max_{1 \leq i \leq q} \left\lceil \frac{m_i}{d_i} \right\rceil,$$

with $\max \varnothing = 0$ when $q = 0$.

Proof. By `PSC-I-T-MODULE-TOP`, $\mathrm{srank}_A(M) = \mathrm{srank}_{A/J}(M/JM)$. Apply `PSC-I-T-SEMISIMPLE-SEED-RANK` to the displayed decompositions of A/J and M/JM, with every $D_i = \Bbbk$. This gives the stated maximum. If $q = 0$, the unital A/J-module M/JM is zero, and `PSC-I-T-MODULE-TOP` gives seed rank zero. □

Formal correspondence. covered. Correct public map uses decomposition of A/J, not A. The supplied algebra equivalence forgets to a ring equivalence, and the module decomposition stays compatible. Specializing the exact radical-top theorem to all D_i=k gives the current statement.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 809–822; proof lines 826–828.

Statement SHA-256. `20c07c2db5ab4c4a956355fe5e45008edff100a4b279dd27a37ddff90871fcbf`

40. Full matrix algebra

Stable ID. `PSC-I-C-FULL-MATRIX`

Kind. corollary

Statement.

For the natural module $S = D^d$ over $M_d(D)$,

$$\mathrm{srank}_{M_d(D)}(S) = 1.$$

More strongly, every nonzero vector of S is cyclic.

Proof. The formula gives $\lceil 1/d \rceil = 1$. Directly, let $v \neq 0$ and choose a coordinate $v_j \neq 0$. For any target $w = (w_i)$, the matrix whose jth column has entries $w_i v_j^{-1}$ and whose other columns vanish sends v to w. □

Formal correspondence. covered. For every division ring and positive matrix size, the natural module has canonical seed rank one and every nonzero vector generates. The proof uses entries w_i / v_j with the inverse on the correct right side. The positive-size convention of the surrounding natural-simple-module discussion is essential: size zero has seed rank zero. This guard remains explicit in Lean and is tested, not silently discarded.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 830–837; proof lines 841–843.

Statement SHA-256. `f22f88ae7b9fe79b50f6c82420d301d784c941d6340fc700723841d724826055`

41. Multiplicity obstruction

Stable ID. `PSC-I-C-MULTIPLICITY-OBSTRUCTION`

Kind. corollary

Statement.

For the diagonal action of $M_d(D)$ on $S^{\oplus m}$,

$$\boxed{\operatorname{srank}_{M_d(D)}(S^{\oplus m}) = \left\lceil \frac{m}{d} \right\rceil .}$$

Proof. Apply `PSC-I-T-SEMISIMPLE-SEED-RANK` to the single simple type with multiplicity m. □

Formal correspondence. covered. For arbitrary division rings and positive block size, the actual shared matrix action on m natural-module copies has canonical rank ceil(m/d). The rectangular-matrix, natural-column-product and finite-direct-sum formulations are connected by actual module equivalences. Zero multiplicity is included. This is not the independent-factor action, whose bound is a maximum across distinct factors.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 845–855; proof lines 859–861.

Statement SHA-256. `df87d9465fd05e359ca1a96ba5b69e501f5e6b9b93b2d4e1b555a84476ec84ef`

42. Primary decomposition for polynomial modules

Stable ID. `PSC-I-CI-PID-PRIMARY-DECOMPOSITION`

Kind. classicalinput

Statement.

Every module X over $\mathbb{k}[t]$ with finite dimension over the coefficient field decomposes as

$$X = \bigoplus_{p \in \operatorname{Irr}(\mathbb{k}[t])} X_p,$$

where $\operatorname{Irr}(\mathbb{k}[t])$ denotes the monic irreducible polynomials, only finitely many components are nonzero, and every nonzero X_p is a finite direct sum of modules $\mathbb{k}[t]/(p^e)$.

Formal correspondence. classical input integrated. For an existing polynomial module, multiplication by X constructs its evaluation operator and an actual polynomial-linear equivalence; finite coefficient dimension gives torsion. PID decomposition constructs positive elementary divisors, while actual

intrinsic primary submodules are proved equal to ideal primary components. PolynomialMonic proves a bijection between monic irreducibles and nonzero prime ideals via p↦(p), deriving injectivity from monicity and surjectivity from a normalized PID prime generator. It reindexes the internal sum and finite nonzero support to precisely the manuscript index set. intrinsic_primary_finite_power_sum is a literal finite DirectSum of k[X]/(p^e), with positive e, for each specified irreducible p.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 883–890.

Statement SHA-256. `446dc677476b213cc4daac554de2b3f15ab816c9d0d6eeff98a67965eea843a9`

43. Single-operator seed rank

Stable ID. `PSC-I-T-SINGLE-OPERATOR`

Kind. theorem

Statement.

Let X have finite dimension over $\Bbbk$ and let $T \in \mathrm{End}_{\Bbbk}(X)$. Regard X as a module over $\Bbbk[t]$ by $t \cdot x = Tx$. For every monic irreducible p, let X_p be the primary component for p and set

$$\mu_p(T) := \dim_{\Bbbk[t]/(p)}(X_p / p(T)X_p).$$

Then

$$\boxed{\mathrm{srank}_{\Bbbk[T]}(X) = \max_p \mu_p(T).}$$

Moreover, $\mu_p(T)$ is the number of cyclic primary summands for p in the decomposition into elementary divisors of X_p.

Proof. Let the minimal polynomial be

$$m_T(t) = \prod_p p(t)^{e_p}.$$

The acting algebra is

$$\Bbbk[T] \cong \Bbbk[t]/(m_T) \cong \prod_p R_p, \qquad R_p := \Bbbk[t]/(p^{e_p}),$$

by the Chinese remainder theorem. The factor R_p acts on X_p and annihilates every other primary component. Hence the formula for a direct sum with independent actions gives

$$\mathrm{srank}_{\Bbbk[T]}(X) = \max_p \mathrm{srank}_{R_p}(X_p).$$

Each R_p is local with maximal ideal generated by the class of p and residue division field $K_p = \Bbbk[t]/(p)$. By `PSC-I-C-LOCAL-ALGEBRA-FORMULA`,

$$\mathrm{srank}_{R_p}(X_p) = \dim_{K_p} X_p / p(T)X_p = \mu_p(T).$$

Finally, every summand $\Bbbk[t]/(p^e)$ contributes one copy of K_p after quotienting by p, so the dimension equals the number of summands in the elementary divisor decomposition. □

Formal correspondence. covered. The full printed statement is now mapped. The actual power-annihilation submodule is proved equal to the ideal primary component. Its inclusion into V/pV

has exact kernel pX_p and is onto from the internally discharged Artinian hypothesis. The induced quotient equivalence is residue-field linear. Every intrinsic multiplicity is identified with every actual presentation fibre. The grouped factors are proved to be the literal X_p, and any positive primary decomposition of X_p has the same intrinsic count. The presentation-free maximum over all monic irreducibles is proved bounded and attained, and equals both natural-valued and cardinal-valued seed rank. No caller-supplied decomposition or seed-count conclusion is needed by the public maximum theorem. These are noncomputable mathematical constructions. The printed proof uses a different route through local-module/Nakayama machinery; that separately listed machinery is not thereby marked complete.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 893–907; proof lines 914–939.

Statement SHA-256. `e96051e5d8333ea1ba1efe94f70b260ad356a41bc5ecad49525ec1df9c759967`

44. Invariant-factor count

Stable ID. `PSC-I-C-INVARIANT-FACTOR-COUNT`

Kind. corollary

Statement.

If $X \neq 0$ and the rational canonical decomposition is written with nonconstant invariant factors

$$X \cong \bigoplus_{j=1}^{r} \mathbb{k}[t]/(f_j), \qquad f_1 \mid f_2 \mid \cdots \mid f_r,$$

then

$$\boxed{\operatorname{srank}_{\mathbb{k}[T]}(X) = r.}$$

Proof. For each irreducible p, the number of invariant factors divisible by p equals the number of elementary divisors for p. Because $f_1 \mid \cdots \mid f_r$ and $f_1 \neq 1$ after zero summands are removed, some irreducible divisor of f_1 divides all r invariant factors. Hence $\max_p \mu_p(T) = r$. □

Formal correspondence. covered. Complete at the displayed conditional scope: an actual polynomial-module equivalence to the finite product of nonconstant factors and their divisibility chain imply least seed number, natural seed rank and cardinal seed rank equal the factor count. The theorem also handles the empty product. Finite products and finite direct sums represent the same finite decomposition. Existence of an ordered invariant-factor presentation is not asserted by this declaration, and is not an additional hypothesis-free claim of the printed conditional corollary.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 951–963; proof lines 967–969.

Statement SHA-256. `c26c3a48531f1cf7f79090c82c8db185af548fc1137f1c427ac815d1d8120e89`

45. Geometric-multiplicity formula

Stable ID. `PSC-I-C-ALGEBRAICALLY-CLOSED-SINGLE-OPERATOR`

Kind. corollary

Statement.

If $\mathbb{k}$ is algebraically closed, then

$$\boxed{\operatorname{srank}_{\mathbb{k}[T]}(X) = \max_{\lambda \in \operatorname{Spec}(T)} \dim_{\mathbb{k}} \operatorname{Ker}(T - \lambda I).}$$

Proof. The irreducibles are $p = t - \lambda$. On the primary component for λ, the quotient by $(T - \lambda I)X_\lambda$ has dimension equal to the number of Jordan blocks, which is also $\dim \operatorname{Ker}(T - \lambda I)$. □

Formal correspondence. covered. Complete at the finite-dimensional algebraically closed scope inherited from the single-operator theorem. The actual intrinsic multiplicity for X-C(a) is identified with the dimension of the actual eigenspace, without diagonalizability. The canonical natural and cardinal seed ranks equal the maximum over the actual spectrum. Off-spectrum dimensions are zero, the empty-spectrum/zero-space maximum is defined as zero, and the spectral maximum is attained on nonzero spaces. No executable spectral algorithm is claimed.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 971–982; proof lines 986–988.

Statement SHA-256. `babe43b45ebd6abb23a0e31ed41c0918a4babb97578847c852f2c9d11e69515f`

46. Cyclic-matrix criterion

Stable ID. `PSC-I-C-CYCLIC-MATRIX`

Kind. corollary

Statement.

For $X \neq 0$ of finite dimension, the following are equivalent:

(i) T has a cyclic vector.

(ii) $\operatorname{srank}_{\mathbb{k}[T]}(X) = 1$.

(iii) Every nonzero primary component for p is cyclic.

(iv) The rational canonical form has one invariant factor.

(v) The minimal polynomial equals the characteristic polynomial.

(vi) After scalar extension to an algebraic closure, every eigenvalue has geometric multiplicity one.

Proof. The equivalence of (i) and (ii) is `PSC-I-P-CYCLIC-CHARACTERIZATION`. By `PSC-I-T-SINGLE-OPERATOR`, (ii) holds exactly when every $\mu_p(T)$ belonging to a nonzero primary component equals one, which is (iii). Relatively prime primary cyclic modules combine into one cyclic module by the Chinese remainder theorem, so (iii) is equivalent to (iv). In rational canonical form, one invariant factor means that the unique invariant factor is both the minimal and characteristic polynomial, proving (iv)⇔(v). Scalar extension preserves the equality of minimal and characteristic polynomials. Over an algebraic closure, that equality is equivalent to one Jordan block for each eigenvalue, which is (vi). □

Formal correspondence. covered. Block 06 closes the exact six-way criterion for a nonzero finite-dimensional operator over an arbitrary field. The final List.TFAE includes actual cyclicity, canonical seed rank one, cyclicity of every nonzero intrinsic primary component, an actual monic positive-degree one-invariant-factor presentation, minpoly=charpoly, and one-dimensional eigenspaces at every spectral value of the literal tensor extension to AlgebraicClosure k. Minimal-polynomial preservation and cyclicity preservation/descent are proved, not supplied as hypotheses. The one-factor presentation has a constructed basis in which the original operator is the standard companion matrix; its factor is proved to be the minimal polynomial. A separate theorem proves that any actual nonconstant invariant-factor chain has exactly one factor iff the operator is cyclic. This is a mathematical existence and equivalence theorem, not an executable normal-form algorithm or a full sorted invariant-factor construction for arbitrary noncyclic operators. The nonzero-space guard is retained in the six-way statement. Legacy conditional wrappers are preserved unchanged for compatibility, but the new public criterion needs no decomposition, multiplicity, polynomial-preservation, spectral, or seed-number certificates.

Source locator. `editable/traditional/02_foundations.tex`; statement lines 990–1001; proof lines 1005–1007.

Statement SHA-256. `d418194e254211c8f964adf5796418ad7aaff14e630caffc05cb8d935de9803a`

A.2 Packets and transfer graphs

47. Finite packet system

Stable ID. `PSC-I-D-PACKET-SYSTEM`

Kind. definition

Statement.

A finite packet system on a vector space X over $\mathbb{k}$ is a family

$$\mathcal{P} = (P_\alpha)_{\alpha \in I} \subseteq \mathrm{End}_{\mathbb{k}}(X)$$

satisfying

$$P_\alpha^2 = P_\alpha, \qquad P_\alpha P_\beta = 0 \quad (\alpha \neq \beta), \qquad \sum_{\alpha \in I} P_\alpha = I_X.$$

Set $X_\alpha = P_\alpha X$. Zero packet spaces may be removed without changing any identity. No orthogonality is assumed unless stated.

Formal correspondence. definition represented. LinearProjectorFamily encodes the actual finite completeness, idempotence and annihilation laws.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 10–25.

Statement SHA-256. `251dfa1ba22ba22eaf53fdcc7880aa1fe05933738f362ef3f6ae72583f539459`

48. Packet support and packet-pure vectors

Stable ID. `PSC-I-D-PACKET-SUPPORT`

Kind. definition

Statement.

For a finite packet system $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ and $x \in X$, define

$$\mathrm{Supp}_{\mathcal{P}}(x) := \{\alpha \in I : P_\alpha x \neq 0\}.$$

The vector x is *packet pure* if $|\mathrm{Supp}_{\mathcal{P}}(x)| \leq 1$. A finite family is packet pure if each of its members is packet pure, although different members may lie in different packets.

Formal correspondence. definition represented. packetSupport is exactly {i | P_i x≠0}. PacketPure is Set.Subsingleton of that support, equivalent for finite support to cardinality at most one. packetPure_iff_zero_or_range identifies purity with zero or membership in one actual packet range; a pure family is pointwise this predicate.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 28–37.

Statement SHA-256. `d95377c31eb89901afc468fd8f0ba293fee6865b0ae3b22eff99df183cc9eb9e`

49. Finite packet aggregation

Stable ID. `PSC-I-L-PACKET-AGGREGATION`

Kind. lemma

Statement.

Let $\sigma : I \twoheadrightarrow J$ be a surjection and define

$$Q_j := \sum_{\sigma(\alpha)=j} P_\alpha.$$

Then $(Q_j)_{j\in J}$ is a finite packet system and

$$Q_j X = \bigoplus_{\sigma(\alpha)=j} X_\alpha.$$

Proof. Pairwise annihilation of the P_α gives

$$Q_j^2 = Q_j, \qquad Q_j Q_k = 0 \quad (j \neq k).$$

Summing over j recovers $\sum_\alpha P_\alpha = I$. The range statement follows from the directness forced by pairwise annihilation. □

Formal correspondence. covered. Q_j is the actual filtered sum of P_i. Computing P_a Q_j proves the laws. Actual range analysis/synthesis over the literal fibre subtype are inverse, and the finite product/direct-sum equivalence gives the printed conclusion.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 45–55; proof lines 59–67.

Statement SHA-256. `504cbd4eec7a5b5e416f91dcfd13e06586e5c1360990ed480424f4a85dfb9195`

50. Packet analysis and synthesis

Stable ID. `PSC-I-D-ANALYSIS-SYNTHESIS`

Kind. definition

Statement.

For a finite packet system, define

$$\mathcal{A}_\mathcal{P} : X \longrightarrow \bigoplus_{\alpha\in I} X_\alpha, \qquad x \longmapsto (P_\alpha x)_\alpha,$$

and

$$\mathcal{S}_\mathcal{P} : \bigoplus_{\alpha\in I} X_\alpha \longrightarrow X, \qquad (x_\alpha)_\alpha \longmapsto \sum_\alpha x_\alpha.$$

Formal correspondence. definition represented. Analysis and synthesis use genuine projector ranges, not unconstrained packet slots.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 69–83.

Statement SHA-256. `f364fcc94b357d761a0bb880e5a2d5761fe7bfc687b768b0f4f7cc261a5ce820`

51. Exact packet-coordinate isomorphism

Stable ID. `PSC-I-T-EXACT-PACKET-COORDINATES`

Kind. theorem

Statement.

For every finite packet system,

$$\boxed{\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}} = I_X, \qquad \mathcal{A}_{\mathcal{P}}\mathcal{S}_{\mathcal{P}} = I_{\oplus X_\alpha}.}$$

Hence

$$\boxed{X \cong \bigoplus_{\alpha \in I} X_\alpha.}$$

Proof. For $x \in X$,

$$\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}}x = \sum_\alpha P_\alpha x = x.$$

If $x_\alpha \in X_\alpha$, then $P_\beta x_\alpha = \delta_{\alpha\beta}x_\alpha$, so

$$\mathcal{A}_{\mathcal{P}}\mathcal{S}_{\mathcal{P}}(x_\alpha)_\alpha = \left(P_\beta \sum_\alpha x_\alpha\right)_\beta = (x_\beta)_\beta.$$

Thus analysis and synthesis are inverse linear maps. □

Formal correspondence. covered. Both inverse identities and the linear equivalence are proved.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 86–100; proof lines 105–121.

Statement SHA-256. `d50988e4b39d3f37422e1d0c2704aca3b8acd89beca5afd9d07ad6bc326fc24f`

52. Packetwise equality

Stable ID. `PSC-I-C-PACKETWISE-EQUALITY`

Kind. corollary

Statement.

For $x, y \in X$,

$$x = y \quad \Longleftrightarrow \quad P_\alpha x = P_\alpha y \text{ for every } \alpha.$$

Proof. The forward implication is immediate. For the reverse implication, sum the packet equalities and use $\sum_\alpha P_\alpha = I$. □

Formal correspondence. covered. Exact equality iff every projected coordinate agrees.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 128–136; proof lines 140–142.

Statement SHA-256. `61919a93863e4ac0a41653a394e44cd8f701eac05335283ac8cf73983da43859`

53. Universal mapping property of finite packet coordinates

Stable ID. `PSC-I-C-PACKET-UNIVERSAL-PROPERTY`

Kind. corollary

Statement.

Let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ be a finite packet system on X, and let Y be a vector space over $\mathbb{k}$.

1. Every family of linear maps $f_\alpha : X_\alpha \to Y$ determines a unique linear map $f : X \to Y$ with $f|_{X_\alpha} = f_\alpha$, namely
$$f = \sum_\alpha f_\alpha P_\alpha.$$
2. Every family of linear maps $g_\alpha : Y \to X_\alpha$ determines a unique linear map $g : Y \to X$ with $P_\alpha g = g_\alpha$, namely
$$g = \sum_\alpha g_\alpha.$$

Equivalently,

$$\mathrm{Hom}(X, Y) \cong \prod_\alpha \mathrm{Hom}(X_\alpha, Y), \qquad \mathrm{Hom}(Y, X) \cong \prod_\alpha \mathrm{Hom}(Y, X_\alpha),$$

where the products are finite.

Proof. For $x = \sum_\alpha P_\alpha x$, any extension of the f_α must satisfy

$$f(x) = \sum_\alpha f_\alpha(P_\alpha x),$$

which proves both existence and uniqueness of the first formula. For the second, the sum $g = \sum_\alpha g_\alpha$ satisfies

$$P_\beta g = \sum_\alpha P_\beta g_\alpha = g_\beta$$

because $g_\alpha(Y) \subseteq X_\alpha$. Uniqueness follows from $\sum_\alpha P_\alpha = I$. □

Formal correspondence. covered. Outgoing extension is the finite sum of f_i after actual rangeRestrict projections. Incoming assembly is actual rangeSynthesis after LinearMap.pi. Packet laws give restrictions/projections and completeness gives uniqueness. These universal properties induce the displayed Hom equivalences.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 146–166; proof lines 171–183.

Statement SHA-256. `3136b91ef330400f72205441080d679179476d4620287e1bd3658a36dca7a4be`

54. Tensor-product packet systems

Stable ID. `PSC-I-T-TENSOR-PACKET-SYSTEM`

Kind. theorem

Statement.

Let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ be a finite packet system on X, and let $\mathcal{Q} = (Q_\gamma)_{\gamma \in J}$ be a finite packet system on Y. Then

$$\mathcal{P} \boxtimes \mathcal{Q} := (P_\alpha \otimes Q_\gamma)_{(\alpha,\gamma) \in I \times J}$$

is a finite packet system on the algebraic tensor product $X \otimes_{\mathbb{k}} Y$, with packet spaces

$$(P_\alpha \otimes Q_\gamma)(X \otimes Y) = X_\alpha \otimes Y_\gamma.$$

Hence

$$\boxed{X \otimes Y \cong \bigoplus_{(\alpha,\gamma) \in I \times J} X_\alpha \otimes Y_\gamma.}$$

Proof. For every pair of packet indices,

$$(P_\alpha \otimes Q_\gamma)^2 = P_\alpha^2 \otimes Q_\gamma^2 = P_\alpha \otimes Q_\gamma.$$

If $(\alpha, \gamma) \neq (\beta, \delta)$, then either $P_\alpha P_\beta = 0$ or $Q_\gamma Q_\delta = 0$, so the corresponding tensor idempotents annihilate each other. Finally,

$$\sum_{\alpha,\gamma} P_\alpha \otimes Q_\gamma = \left(\sum_\alpha P_\alpha \right) \otimes \left(\sum_\gamma Q_\gamma \right) = I_X \otimes I_Y.$$

The range is generated by pure tensors $P_\alpha x \otimes Q_\gamma y$, hence equals $X_\alpha \otimes Y_\gamma$. Exact reconstruction follows from `PSC-I-T-EXACT-PACKET-COORDINATES`. □

Formal correspondence. covered. Actual TensorProduct.map projectors satisfy all laws by tensor induction. Tensor products of range inclusions have the tensor products of projections as left inverses; range_map identifies the actual packet range. This proves the canonical embedded interpretation of the printed range equality. Exact coordinates then give the literal DirectSum.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 188–210; proof lines 215–232.

Statement SHA-256. `afea7128594f24b1249382b028d823051a35d9bd3a3473c247a7b9bd63c5ba47`

55. Dual packet systems

Stable ID. `PSC-I-P-DUAL-PACKET-SYSTEM`

Kind. proposition

Statement.

Let X be finite dimensional and let $\mathcal{P} = (P_\alpha)$ be a finite packet system. The algebraic dual maps

$$P_\alpha^* : X^* \to X^*, \qquad P_\alpha^*(\varphi) = \varphi \circ P_\alpha,$$

form a finite packet system on X^*. Its packet with index α is canonically isomorphic to X_α^*, and

$$X^* \cong \bigoplus_\alpha X_\alpha^*.$$

Proof. Because dualization reverses products,

$$P_\alpha^* P_\beta^* = (P_\beta P_\alpha)^* = \delta_{\alpha\beta} P_\alpha^*.$$

Also $\sum_\alpha P_\alpha^* = I_{X^*}$. Thus the dual maps form a packet system. Restriction gives a map from $P_\alpha^* X^*$ to X_α^*. It is injective because a covector in the range of P_α^* vanishes on every other packet, and it is surjective because any covector on X_α extends to X through the direct sum decomposition. □

Formal correspondence. covered. Actual dualMap means precomposition. Composition reverses indices and orthogonality uses the reversed inequality correctly. Restriction and precomposition with the actual packet projection are inverse, giving the stated packet duals and literal finite DirectSum.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 234–246; proof lines 251–258.

Statement SHA-256. `ade34fd8142ece280ab2ae85913a943cb28a3661978659fde4a644eb5018d119`

56. Finite Banach packet isomorphism

Stable ID. `PSC-I-T-BANACH-PACKET-ISOMORPHISM`

Kind. theorem

Statement.

Let X be Banach and let $(P_\alpha)_{\alpha \in I}$ be a finite family of bounded pairwise annihilating idempotents with sum I_X. Then every range $X_\alpha = P_\alpha X$ is closed. For any finite product norm equivalent to the maximum norm on $\bigoplus_\alpha X_\alpha$, packet analysis and synthesis are bounded inverse operators.

Proof. The range of a bounded idempotent is its space of fixed points,

$$P_\alpha X = \operatorname{Ker}(I - P_\alpha),$$

which is closed. Because I is finite, analysis and synthesis are bounded for the maximum and sum norms, respectively, and therefore for every equivalent finite product norm. Their inverse identities are algebraic and were proved in `PSC-I-T-EXACT-PACKET-COORDINATES`. □

Formal correspondence. covered. The actual finite projectors are continuous and explicitly bounded as IsBoundedLinearMap over an arbitrary NormedField, including a trivial valuation. Individual bounds

produce one finite uniform bound; analysis has that bound and synthesis has the finite-cardinality bound. Any coordinate norm equivalent to the canonical maximum norm is represented by a linear identification and its literal two-sided norm inequalities. Composition gives bounded actual analysis and synthesis, both inverse identities, and closed packet ranges. No conclusion-shaped hypotheses occur.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 267–270; proof lines 274–280.

Statement SHA-256. `3fd1f470c1a5dcd6273739c6f6cb95e5fbb8f8c95e4fc4c76e010430a72afa3e`

57. Packet-coordinate condition number

Stable ID. `PSC-I-D-PACKET-CONDITION-NUMBER`

Kind. definition

Statement.

Fix a finite product norm on $\bigoplus_\alpha X_\alpha$ equivalent to the maximum norm, as in `PSC-I-T-BANACH-PACKET-ISOMORPHISM`. Define

$$\kappa(\mathcal{P}) := \|\mathcal{A}_\mathcal{P}\|\,\|\mathcal{S}_\mathcal{P}\|.$$

Formal correspondence. definition represented. The definition is the product of the ordinary operator norms of the actual bounded analysis and synthesis. The generic bounded additive homomorphism norm is defined as the infimum of nonnegative valid operator bounds; linearity is retained by ofBoundedLinear. Equivalent-norm coordinates have explicit actual-map equalities.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 282–290.

Statement SHA-256. `7c67c989c536ab3eecad8fdf8c19b0ed6eef7c164c569d69a0e04de101b2460d`

58. Conditioned coordinate transport

Stable ID. `PSC-I-P-CONDITIONED-COORDINATES`

Kind. proposition

Statement.

For a finite Banach packet system with $X \neq 0$,

$$\kappa(\mathcal{P}) \geq 1.$$

If $T \in \mathcal{B}(X)$ and

$$[T]_\mathcal{P} := \mathcal{A}_\mathcal{P} T \mathcal{S}_\mathcal{P},$$

then

$$\boxed{\|[T]_\mathcal{P}\| \leq \kappa(\mathcal{P})\|T\|, \qquad \|T\| \leq \kappa(\mathcal{P})\|[T]_\mathcal{P}\|.}$$

Proof. Since $I = \mathcal{S}_\mathcal{P}\mathcal{A}_\mathcal{P}$,

$$1 = \|I\| \leq \|\mathcal{S}_\mathcal{P}\|\|\mathcal{A}_\mathcal{P}\|.$$

The first displayed estimate follows from submultiplicativity. The inverse identity

$$T = \mathcal{S}_\mathcal{P}[T]_\mathcal{P}\mathcal{A}_\mathcal{P}$$

gives the second estimate. □

Formal correspondence. covered. For nonzero X the identity has norm one. The actual inverse relation S∘A=id and operator norm submultiplicativity give $\kappa \geq 1$. The coordinate operator is literally A∘T∘S, and S∘[T]∘A=T gives the reverse estimate. The final wrapper constructs both bounded actual coordinate maps for every equivalent finite-product norm and proves both inequalities for every bounded linear T over any NormedField.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 293–313; proof lines 317–327.

Statement SHA-256. `6799946dd2c88a65a635b418c7d0a74e94a9af4d1dc88df66cdd3b05c4ebc180`

59. Completion preserves a finite packet split

Stable ID. `PSC-I-C-COMPLETION-PACKET-SPLIT`

Kind. corollary

Statement.

Let M be a dense normed subspace of a Banach space X. Suppose every bounded P_α leaves M invariant and the restricted projectors form a packet system on M. Then

$$P_\alpha X = \overline{P_\alpha M}$$

and the packet decomposition of M extends uniquely to the completion X.

Proof. For $x \in X$, choose $m_n \in M$ with $m_n \to x$. Continuity gives $P_\alpha m_n \to P_\alpha x$, so $P_\alpha X \subseteq \overline{P_\alpha M}$. The reverse inclusion follows because $P_\alpha X$ is closed and contains $P_\alpha M$. The bounded extensions remain pairwise annihilating idempotents whose finite sum is the identity. □

Formal correspondence. covered. Over an arbitrary NormedField, the actual dense-submodule restrictions determine the ambient continuous projector identities by equality on a dense set. Each ambient range equals closure of the corresponding dense-submodule image, and two continuous ambient projector extensions agreeing on the dense submodule coincide. This is stronger than the bounded-input manuscript statement; boundedness of the given ambient projectors is available from the printed premise and is not needed for the density/uniqueness argument.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 334–341; proof lines 345–347.

Statement SHA-256. `67202410fc4580e865ea492dba6cf8dff7e0f296d89ac886c96188a0169a6b70`

60. Unconditional packet sum space

Stable ID. PSC-I-D-UNCONDITIONAL-PACKET-SPACE

Kind. definition

Statement.

Let $(X_n)_{n\ge 1}$ be closed subspaces of a real or complex Banach space X. Define

$$\bigoplus_{n\ge 1}^{u} X_n$$

to be the space of sequences $x = (x_n)$ with $x_n \in X_n$ for which $\sum_n x_n$ converges unconditionally in X. Equip it with

$$\|x\|_u := \sup_{F\in\mathbb{N}} \sup_{|a_n|\le 1} \left\| \sum_{n\in F} a_n x_n \right\|.$$

Formal correspondence. definition represented. The carrier is exactly sequences x_n in the specified closed subspaces for which Summable x holds; mathlib Summable is the finite-subset net (unconditional summation). The norm is exactly sSup of norms of finite sums with coefficients in the closed scalar unit ball. Type synonyms avoid inherited product topology. Finiteness is derived, not assumed. RCLike covers real and complex scalars as stated.

Source locator. editable/traditional/03_packets_and_graphs.tex; statement lines 352–366.

Statement SHA-256. bcbeba58302101658c66251d7b338a6a3932fb07b059e77931ffd063b5303b4f

61. Completeness of the unconditional packet sum

Stable ID. PSC-I-L-UNCONDITIONAL-COMPLETENESS

Kind. lemma

Statement.

The normed space $\bigoplus_{n\ge 1}^{u} X_n$ in PSC-I-D-UNCONDITIONAL-PACKET-SPACE is Banach.

Proof. Let $x^{(k)} = (x_n^{(k)})$ be Cauchy in $\|\cdot\|_u$. For every fixed n,

$$\|x_n^{(k)} - x_n^{(\ell)}\| \le \|x^{(k)} - x^{(\ell)}\|_u,$$

so $x_n^{(k)}$ converges to some $x_n \in X_n$ because X_n is closed. Given $\varepsilon > 0$, choose k_0 such that

$$\|x^{(k)} - x^{(\ell)}\|_u < \varepsilon \qquad (k, \ell \ge k_0).$$

For finite F and $|a_n| \le 1$, pass $\ell \to \infty$ in the finite sum to obtain

$$\left\| \sum_{n\in F} a_n (x_n^{(k)} - x_n) \right\| \le \varepsilon \qquad (k \ge k_0).$$

Hence $\|x^{(k)} - x\|_u \le \varepsilon$ once x is known to be unconditionally summable.

Choose such a $k \geq k_0$. Since $x^{(k)}$ is unconditionally summable, there is a finite F_0 such that every finite $F \subseteq \mathbb{N} \setminus F_0$ and every $|a_n| \leq 1$ satisfy

$$\left\| \sum_{n \in F} a_n x_n^{(k)} \right\| < \varepsilon.$$

The preceding uniform estimate then gives

$$\left\| \sum_{n \in F} a_n x_n \right\| < 2\varepsilon.$$

Thus the tails of $\sum_n x_n$ are uniformly small under bounded scalar changes, which is the unconditional Cauchy criterion. Therefore $x \in \bigoplus^u X_n$, and the earlier estimate proves $x^{(k)} \to x$ in $\| \cdot \|_u$. □

Formal correspondence. covered. Uniform boundedness on the complete continuous dual is justified by finite-dimensional real absolute summability of each scalar image; the isometric bidual inclusion yields finite multiplier bounds. The signed norm controls each coordinate; coordinate limits stay in closed subspaces. Each fixed finite multiplier estimate passes to the limit, uniformly in the finite set and coefficients. Unweighted tails prove Summable for the limit, and the same uniform estimate gives convergence in the signed norm. Metric completeness is then obtained without a projector hypothesis.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 369–372; proof lines 376–405.

Statement SHA-256. `339adef1c7737cb8adc8a4bc76b3a5a2f95a575ffb576a34404d8da60bfe0b62`

62. Countable unconditional packet isomorphism

Stable ID. `PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS`

Kind. theorem

Statement.

Let X be a real or complex Banach space and let $(P_n)_{n \geq 1} \subseteq \mathcal{B}(X)$ satisfy

$$P_n P_m = \delta_{nm} P_n$$

and

$$x = \sum_{n \geq 1} P_n x$$

unconditionally for every $x \in X$. Assume

$$K := \sup_{F \in \mathbb{N}} \sup_{|a_n| \leq 1} \left\| \sum_{n \in F} a_n P_n \right\| < \infty.$$

Then analysis and synthesis define inverse Banach space isomorphisms

$$\boxed{X \cong \bigoplus_{n \geq 1}^{u} P_n X.}$$

More precisely,

$$\|\mathcal{A}_{\mathcal{P}}\| \le K, \qquad \|\mathcal{S}_{\mathcal{P}}\| \le 1.$$

Proof. Each $P_nX = \operatorname{Ker}(I - P_n)$ is closed. For $x \in X$,

$$\|\mathcal{A}_{\mathcal{P}}x\|_u = \sup_{F,a} \left\| \sum_{n\in F} a_n P_n x \right\| \le K\|x\|.$$

For $y = (y_n) \in \bigoplus^u P_nX$, define

$$\mathcal{S}_{\mathcal{P}}y = \sum_n y_n.$$

Since the finite partial sums with every coefficient equal to one are among those used in $\|y\|_u$,

$$\|\mathcal{S}_{\mathcal{P}}y\| \le \|y\|_u.$$

The assumed expansion gives $\mathcal{S}_{\mathcal{P}}\mathcal{A}_{\mathcal{P}}x = x$. Conversely, boundedness of P_m permits termwise application to the unconditionally convergent series:

$$P_m\mathcal{S}_{\mathcal{P}}y = \sum_n P_m y_n = y_m.$$

Thus $\mathcal{A}_{\mathcal{P}}\mathcal{S}_{\mathcal{P}} = I$. Completeness of the coordinate space is PSC-I-L-UNCONDITIONAL-COMPLETENESS. □

Formal correspondence. covered. ofSupBound constructs K as the literal supremum of finite multiplier operator norms from BddAbove. The carriers use Summable sequences in actual projector ranges. Continuous projectors commute with the sum, so annihilation proves both inverse identities. The literal signed norm yields analysis bound K and synthesis bound 1. A proved linear isometry identifies the projector-specific carrier and norm with the general ClosedPackets.Space. Completeness follows either from the general theorem or from the constructed uniform equivalence with Banach X.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 407–436; proof lines 441–467.

Statement SHA-256. `7579699c5a34468a6f9df136e7c897641c5409e5f5e597cd849348bddc92a469`

63. Uniform boundedness

Stable ID. `PSC-I-R-UNIFORM-BOUNDEDNESS`

Kind. remark

Statement.

If every expansion $\sum_n P_n x$ is unconditional, the operators obtained from finite partial sums with signs form a pointwise bounded family. The uniform boundedness principle therefore yields a finite constant K. We retain this constant explicitly in PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS because later estimates use it to bound the coordinate map.

Formal correspondence. documentary. A uniform-boundedness explanation. No instance for the signed packet partial-sum family is mapped.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 469–472.

Statement SHA-256. `2ae822d9b50496c6462b94007b5ff1a8d083e7109e6da7d0c2453ca40b80373c`

64. Infinite packet sums

Stable ID. `PSC-I-B-INFINITE-PACKET-TOPOLOGY`

Kind. boundary

Statement.

For an infinite family, an identity such as $\sum_\alpha P_\alpha = I$ requires a declared convergence mode and the corresponding uniform boundedness statement. Pointwise, strong, norm, weak, and unconditional convergence are not interchangeable.

Formal correspondence. scope boundary. Convergence modes remain separate; finite identities are not used as infinite sums.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 475–478.

Statement SHA-256. `305cbc8493dffcc2407698c4df7a3f54fecc76bb57e10c75390b0cedc5bce53d`

65. Transfer block

Stable ID. `PSC-I-D-TRANSFER-BLOCK`

Kind. definition

Statement.

For $T \in \mathrm{End}_{\Bbbk}(X)$ and a finite packet system $\mathcal{P}$, define

$$T_{\beta\alpha} := P_\beta T P_\alpha \in \mathrm{Hom}_{\Bbbk}(X_\alpha, X_\beta).$$

Formal correspondence. definition represented. Ambient compositions and their genuinely range-restricted packet Hom maps are both encoded.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 492–499.

Statement SHA-256. `1b27b671e8c43628db2f3e157df05e01a63658c32e64a6ae2a94fdc69fd5ad73`

66. Exact operator reconstruction

Stable ID. `PSC-I-T-OPERATOR-RECONSTRUCTION`

Kind. theorem

Statement.

Every $T \in \mathrm{End}_{\Bbbk}(X)$ satisfies

$$\boxed{T = \sum_{\alpha,\beta \in I} T_{\beta\alpha}.}$$

Proof. Insert the finite packet identity on both sides:

$$T = \left(\sum_\beta P_\beta\right) T \left(\sum_\alpha P_\alpha\right) = \sum_{\alpha,\beta} P_\beta T P_\alpha.$$

□

Formal correspondence. covered. The existing result is more general: source and target can have different finite packet systems.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 502–508; proof lines 512–522.

Statement SHA-256. `f8c54ea5f0aa3075a42556f7d9b9b29fd9399d88e9de974c6eb098fceb2a7abb`

67. Packet matrix multiplication

Stable ID. `PSC-I-T-BLOCK-MULTIPLICATION`

Kind. theorem

Statement.

For $S, T \in \mathrm{End}_{\Bbbk}(X)$,

$$\boxed{(ST)_{\gamma\alpha} = \sum_{\beta \in I} S_{\gamma\beta} T_{\beta\alpha}.}$$

Proof. Insert $I = \sum_{\beta} P_{\beta}$ between S and T:

$$P_{\gamma} S T P_{\alpha} = \sum_{\beta} P_{\gamma} S P_{\beta} T P_{\alpha}.$$

□

Formal correspondence. covered. The output-first block product is proved for three possibly different modules.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 529–539; proof lines 544–552.

Statement SHA-256. `a7aaf1cde8454ca239c380ca964f5b13f20d39baaf62b7a859caf9a849378314`

68. Packet powers and paths

Stable ID. `PSC-I-C-BLOCK-POWERS`

Kind. corollary

Statement.

For $m \geq 1$,

$$(T^m)_{\beta_m \beta_0} = \sum_{\beta_1, \ldots, \beta_{m-1}} T_{\beta_m \beta_{m-1}} \cdots T_{\beta_1 \beta_0}.$$

The same formula holds for a product of nonidentical operators with the corresponding blocks in temporal order.

Proof. Induct on m using `PSC-I-T-BLOCK-MULTIPLICATION`. For a product of different operators, use the same induction with the specified operator at each step. □

Formal correspondence. covered. The power theorem expands transferBlock(P,P,T^m) using packetPathSum. The independent operatorWord/packetWordSum induction expands every finite list of

possibly different operators, with composition order matching rightmost-acts-first temporal order. Each summation is over actual intermediate packet labels.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 559–571; proof lines 575–577.

Statement SHA-256. `dbaa3f1f55517bd6366bcf856dc8eb6836a642144d805f9bed4899e5bba7038f`

69. Adjoint reversal

Stable ID. `PSC-I-C-ADJOINT-REVERSAL`

Kind. corollary

Statement.

If X is Hilbert, every P_α is orthogonal, and $T \in \mathcal{B}(X)$, then

$$\boxed{(T_{\beta\alpha})^* = (T^*)_{\alpha\beta}.}$$

Proof. Because $P_\alpha^* = P_\alpha$,

$$(P_\beta T P_\alpha)^* = P_\alpha T^* P_\beta.$$

□

Formal correspondence. covered. The result is proved for bounded self-adjoint outer maps, more generally than orthogonal packet projectors.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 579–585; proof lines 589–594.

Statement SHA-256. `8e978a295a29a3eac3d1c4bec1a8e3f4e05a2684fdbce8e80a357aa7891e8f79`

70. Tensor-product transfer blocks

Stable ID. `PSC-I-C-TENSOR-TRANSFER-BLOCKS`

Kind. corollary

Statement.

Let $S \in \mathrm{End}(X)$ and $T \in \mathrm{End}(Y)$, with packet systems $\mathcal{P}$ and $\mathcal{Q}$. Relative to the tensor product packet system of `PSC-I-T-TENSOR-PACKET-SYSTEM`,

$$\boxed{(S \otimes T)_{(\beta,\delta),(\alpha,\gamma)} = S_{\beta\alpha} \otimes T_{\delta\gamma}.}$$

Proof. By definition,

$$\begin{aligned}(S \otimes T)_{(\beta,\delta),(\alpha,\gamma)} &= (P_\beta \otimes Q_\delta)(S \otimes T)(P_\alpha \otimes Q_\gamma) \\ &= (P_\beta S P_\alpha) \otimes (Q_\delta T Q_\gamma) \\ &= S_{\beta\alpha} \otimes T_{\delta\gamma}.\end{aligned}$$

□

Formal correspondence. covered. Actual transfer blocks factor by tensor-map composition in exactly the (target,source) order printed. Both sides are actual endomorphisms of X tensor Y; no factorization witness is an input.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 598–608; proof lines 613–621.

Statement SHA-256. `ec0bc2d2c85cdf55ea2f27003457a653f8f2426fe97dafd6c918cbda54874878`

71. Blockwise intertwining equations

Stable ID. `PSC-I-T-BLOCK-INTERTWINING`

Kind. theorem

Statement.

Let $T \in \mathrm{End}(X)$, $S \in \mathrm{End}(Y)$, let $\mathcal{P} = (P_\alpha)_{\alpha\in I}$ and $\mathcal{Q} = (Q_\gamma)_{\gamma\in J}$ be finite packet systems, and let $U : X \to Y$ be linear. Define

$$T_{\beta\alpha} = P_\beta T P_\alpha, \qquad S_{\gamma\delta} = Q_\gamma S Q_\delta, \qquad U_{\gamma\alpha} = Q_\gamma U P_\alpha.$$

Then

$$\boxed{UT = SU}$$

if and only if, for every $\gamma \in J$ and $\alpha \in I$,

$$\boxed{\sum_{\beta\in I} U_{\gamma\beta} T_{\beta\alpha} = \sum_{\delta\in J} S_{\gamma\delta} U_{\delta\alpha}.}$$

Proof. Apply Q_γ on the left and P_α on the right. Inserting $I_X = \sum_\beta P_\beta$ gives

$$Q_\gamma U T P_\alpha = \sum_\beta Q_\gamma U P_\beta T P_\alpha = \sum_\beta U_{\gamma\beta} T_{\beta\alpha}.$$

Inserting $I_Y = \sum_\delta Q_\delta$ gives

$$Q_\gamma S U P_\alpha = \sum_\delta Q_\gamma S Q_\delta U P_\alpha = \sum_\delta S_{\gamma\delta} U_{\delta\alpha}.$$

Thus $UT = SU$ implies every block equation. Conversely, if every block equation holds, then all blocks of $UT - SU$ vanish. Exact operator reconstruction yields $UT - SU = 0$. □

Formal correspondence. covered. U is any linear map, not an intertwiner structure. Forward direction inserts middle projector sums; reverse direction reconstructs UT and SU from all equal blocks. Distinct source/target index sets are retained.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 623–645; proof lines 650–668.

Statement SHA-256. `558adbc4eead21c3f371918f40b9f462000c7b26a7234864ee44c103e09cf1e6`

72. Full block-algebra representation

Stable ID. `PSC-I-T-BLOCK-ALGEBRA`

Kind. theorem

Statement.

The map

$$\Theta_{\mathcal{P}} : \mathrm{End}_{\Bbbk}(X) \longrightarrow \prod_{\beta,\alpha\in I} \mathrm{Hom}_{\Bbbk}(X_\alpha, X_\beta), \qquad T \longmapsto (T_{\beta\alpha})_{\beta,\alpha},$$

is an algebra isomorphism when the codomain is equipped with matrix multiplication whose entries are operators.

Proof. Injectivity follows from exact reconstruction. Given arbitrary linear maps $B_{\beta\alpha} : X_\alpha \to X_\beta$, define

$$T\left(\sum_\alpha x_\alpha\right) := \sum_{\alpha,\beta} B_{\beta\alpha} x_\alpha.$$

The packet decomposition is unique and the index set is finite, so this defines a linear operator whose blocks are the prescribed maps. Thus $\Theta_{\mathcal{P}}$ is surjective. Compatibility with multiplication is `PSC-I-T-BLOCK-MULTIPLICATION`. □

Formal correspondence. covered. Full dependent packet Hom families and their multiplication are identified with the actual endomorphism algebra.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 675–686; proof lines 691–699.

Statement SHA-256. `a66e4216289ceb5e48d1ccd41d269b656c6c93ed462a47f68cbd0172c9360bda`

73. Propagation of packet support

Stable ID. `PSC-I-C-PACKET-SUPPORT-PROPAGATION`

Kind. corollary

Statement.

Let $T \in \mathrm{End}(X)$ and $x \in X$. Then

$$\boxed{\mathrm{Supp}_{\mathcal{P}}(Tx) \subseteq \left\{\beta : \exists \alpha \in \mathrm{Supp}_{\mathcal{P}}(x) \text{ with } T_{\beta\alpha} \neq 0\right\}.}$$

Equality holds whenever, for every target β in the set on the right, the active contributions

$$(T_{\beta\alpha} P_\alpha x : \alpha \in \mathrm{Supp}_{\mathcal{P}}(x),\ T_{\beta\alpha} \neq 0)$$

have nonzero sum. In particular, equality holds if at most one active source packet feeds each such β and that source coordinate does not lie in the kernel of the corresponding block.

Proof. For every target packet β,

$$P_\beta T x = \sum_{\alpha\in\mathrm{Supp}_{\mathcal{P}}(x)} T_{\beta\alpha} P_\alpha x.$$

If every block in this sum is zero, then the coordinate with index β vanishes. This proves the inclusion. Equality is exactly the assertion that the displayed active sum is nonzero for every target in the set on the right. The final sufficient condition leaves at most one nonzero summand at each target, and kernel avoidance makes that summand nonzero. □

Formal correspondence. covered. The source expands each target coordinate as the finite sum of the actual transfer-block contributions and filters to active sources. This yields the support inclusion; nonzero active sums give equality. The single-active theorem uses activeSources={a} plus nonzero transfer contribution, which is equivalent to at-most-one active source plus kernel avoidance on a target that is active.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 703–720; proof lines 725–733.

Statement SHA-256. `f05b8bf4828b88f6020e2e0b824e228acffcb77d7a514ced31ec90c64e6d4437`

74. Dimension count

Stable ID. `PSC-I-C-DIMENSION-COUNT`

Kind. corollary

Statement.

If X is finite dimensional and $d_\alpha = \dim X_\alpha$, then

$$\dim \mathrm{End}_{\Bbbk}(X) = \sum_{\alpha,\beta} d_\alpha d_\beta = \left(\sum_{\alpha} d_\alpha \right)^2 .$$

Proof. Take dimensions in the block algebra isomorphism and use $\dim \mathrm{Hom}(X_\alpha, X_\beta) = d_\alpha d_\beta$. □

Formal correspondence. covered. The actual End space is linearly equivalent to the heterogeneous packet Hom product. Its finite dimension equals the sum of products of actual packet-range dimensions, and finite distributivity gives the displayed square. Both printed equalities occur in one conjunction.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 736–746; proof lines 750–752.

Statement SHA-256. `85dd27ca5c20eeb56683b4b3d9b9093503e2c3c2fdc4a07c1aaf33eefab855c8`

75. Block-diagonal projection and packet commutant

Stable ID. `PSC-I-P-BLOCK-DIAGONAL-COMMUTANT`

Kind. proposition

Statement.

Define

$$\Delta_{\mathcal{P}}(T) := \sum_{\alpha} P_\alpha T P_\alpha .$$

Then $\Delta_{\mathcal{P}}$ is a linear idempotent on $\mathrm{End}(X)$ whose range is the block diagonal subalgebra. For $T \in \mathrm{End}(X)$, the following are equivalent:

(i) $TP_\alpha = P_\alpha T$ for every α.

(ii) $T_{\beta\alpha} = 0$ whenever $\beta \neq \alpha$.

(iii) $\Delta_{\mathcal{P}}(T) = T$.

Proof. Pairwise annihilation gives $\Delta_{\mathcal{P}}^2 = \Delta_{\mathcal{P}}$ and identifies its range with diagonal block families. If T commutes with all P_α, then for $\beta \neq \alpha$,

$$P_\beta T P_\alpha = P_\beta P_\alpha T = 0.$$

Thus (i) implies (ii). Exact reconstruction shows (ii) implies (iii). If $T = \sum_\alpha P_\alpha T P_\alpha$, then

$$T P_\gamma = P_\gamma T P_\gamma = P_\gamma T,$$

so (iii) implies (i). □

Formal correspondence. covered. diagonalProjection is the actual K-linear map T↦Σ_i P_i T P_i. The proofs derive idempotence, equivalence of commutation with every projector and off-diagonal block vanishing, equivalence to fixed points of this projection, and equality of its actual linear range to the equality-supported subalgebra.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 754–766; proof lines 771–781.

Statement SHA-256. `613238e060365a7c00244f44abb552ada9f54933d8483382f259d3f32843eae5`

76. Support relation and support space

Stable ID. `PSC-I-D-SUPPORT-RELATION`

Kind. definition

Statement.

For a relation $R \subseteq I \times I$, with (β, α) interpreted as an allowed transfer $\alpha \to \beta$, define

$$\mathfrak{M}_R := \{T \in \mathrm{End}(X) : T_{\beta\alpha} = 0 \text{ whenever } (\beta, \alpha) \notin R\}.$$

Formal correspondence. definition represented. OperatorSupported is initially defined via actual restricted packet maps; operatorSupported_iff_ambient proves its exact equivalence to ambient transfer-block vanishing. supportSubmodule is the corresponding actual linear subspace of End(X). Relation dictionary: formal rel(a,b) means manuscript (b,a)∈R, with a source and b target.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 789–797.

Statement SHA-256. `babc2ddcee7669992338d1b7741bd50fab01175a280772ab40df14dcc814ebf3`

77. Support-pattern algebra criterion

Stable ID. `PSC-I-T-SUPPORT-PATTERN-ALGEBRA`

Kind. theorem

Statement.

Assume $X_\alpha \neq 0$ for every $\alpha \in I$. Then:

(i) $I_X \in \mathfrak{M}_R$ if and only if R is reflexive.

(ii) $\mathfrak{M}_R\mathfrak{M}_R \subseteq \mathfrak{M}_R$ if and only if R is transitive.

(iii) $\mathfrak{M}_R$ is a unital subalgebra if and only if R is a preorder.

Proof. The identity has diagonal blocks I_{X_α} and no blocks outside the diagonal, so it belongs to $\mathfrak{M}_R$ exactly when $(\alpha, \alpha) \in R$ for every α.

Assume R is transitive. If a summand

$$S_{\gamma\beta}T_{\beta\alpha}$$

in the (γ, α) block of ST is nonzero, then (γ, β) and (β, α) lie in R. Transitivity gives $(\gamma, \alpha) \in R$, so every forbidden block of ST vanishes.

Conversely, suppose transitivity fails. Choose α, β, γ with

$$(\beta, \alpha) \in R, \qquad (\gamma, \beta) \in R, \qquad (\gamma, \alpha) \notin R.$$

Choose nonzero vectors $u \in X_\alpha$, $v \in X_\beta$, and $w \in X_\gamma$. Choose linear functionals $f \in X_\alpha^*$ and $g \in X_\beta^*$ with $f(u) = g(v) = 1$. Let T have the sole block $x \mapsto f(x)v$ from X_α to X_β, and let S have the sole block $y \mapsto g(y)w$ from X_β to X_γ. Then $S, T \in \mathfrak{M}_R$, but ST has a nonzero forbidden (γ, α) block. This proves necessity. Combining reflexivity and transitivity gives (iii). □

Formal correspondence. covered. The two difficult converses use arbitrary maps between the actual nonzero packet ranges. Identity support is equivalent to reflexivity; multiplication closure is equivalent to transitivity. support_mul_closed_iff transfers the block statement to actual ambient operators, and support_pattern_algebra_criterion characterizes existence of a Subalgebra whose carrier is precisely that support space.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 800–808; proof lines 812–830.

Statement SHA-256. `fcd8af3b6718c7460c4e58639b8e942c24ede23b8673f9811f193cab4ccd3b20`

78. $*$-closed bounded support patterns

Stable ID. `PSC-I-C-SUPPORT-PATTERN-STAR`

Kind. corollary

Statement.

Let X be Hilbert, let the packet projectors be orthogonal, assume $X_\alpha \neq 0$ for every $\alpha \in I$, and put

$$\mathfrak{M}_R^{\mathrm{b}} := \mathfrak{M}_R \cap \mathcal{B}(X).$$

Then $\mathfrak{M}_R^{\mathrm{b}}$ is closed under adjoints if and only if R is symmetric. Hence $\mathfrak{M}_R^{\mathrm{b}}$ is a unital $*$ subalgebra of $\mathcal{B}(X)$ if and only if R is an equivalence relation.

Proof. Adjoint reversal sends an allowed (β, α) block to an (α, β) block. Symmetry is therefore sufficient. If symmetry fails, choose a nonzero map of rank one in an allowed nonsymmetric block. Its adjoint occupies the forbidden reverse block, proving necessity. A reflexive, transitive, symmetric relation is an equivalence relation. □

Formal correspondence. covered. Supported uses literal continuous transfer blocks and is proved equivalent to the underlying algebraic support predicate. Converse symmetry and transitivity use actual bounded rank-one operators between nonzero ranges. Identity support forces reflexivity. Sufficiency constructs an actual StarSubalgebra. RCLike covers the real and complex Hilbert settings; no finite-dimensional restriction or blanket bounded-block assumption is used.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 832–839; proof lines 843–845.

Statement SHA-256. `89dfdc356ea3dbbf9ee8018c66d8307f445d19078e0d7226372aae44c1872333`

79. Preorder support algebra decomposition

Stable ID. `PSC-I-T-PREORDER-ALGEBRA-DECOMPOSITION`

Kind. theorem

Statement.

Let R be a preorder on the finite set I. Define

$$\alpha \sim \beta \quad \Longleftrightarrow \quad (\beta, \alpha) \in R \text{ and } (\alpha, \beta) \in R.$$

Let $\overline{I} = I/\!\sim$, which carries the induced partial order, and put

$$X_C := \bigoplus_{\alpha \in C} X_\alpha.$$

Let $\mathfrak{N}_R \subseteq \mathfrak{M}_R$ consist of operators whose blocks vanish whenever source and target belong to the same equivalence class. Then:

(i) $\mathfrak{N}_R$ is a nilpotent ideal on both sides.

(ii) If h is the maximal number of strict steps in a chain of $\overline{I}$, then

$$\mathfrak{N}_R^{h+1} = 0.$$

(iii) The map to the diagonal classes induces an algebra isomorphism

$$\boxed{\mathfrak{M}_R/\mathfrak{N}_R \cong \prod_{C\in\overline{I}} \mathrm{End}_{\Bbbk}(X_C).}$$

Proof. A nonzero block in $\mathfrak{N}_R$ moves from one equivalence class to a strictly larger class in the quotient partial order. The block multiplication law shows that products concatenate such moves. Therefore left or right multiplication by an element of $\mathfrak{M}_R$ preserves the property of having no blocks within a single class, so $\mathfrak{N}_R$ is an ideal on both sides. A product of $h+1$ elements of $\mathfrak{N}_R$ would require a strict chain of $h+1$ steps in $\overline{I}$, which does not exist. This proves nilpotence.

Let $P_C = \sum_{\alpha\in C} P_\alpha$. Define

$$\rho : \mathfrak{M}_R \longrightarrow \prod_C \mathrm{End}(X_C), \qquad \rho(T) = (P_C T P_C)_C.$$

If an intermediate class D contributes to $P_C S T P_C$, then the preorder contains transfers $C \to D$ and $D \to C$, forcing $C = D$ in the quotient poset. Hence

$$P_C S T P_C = (P_C S P_C)(P_C T P_C),$$

so ρ is an algebra homomorphism. Every collection of endomorphisms of the X_C defines an allowed operator that is block diagonal with respect to the classes, so ρ is surjective. Its kernel is exactly $\mathfrak{N}_R$. The first isomorphism theorem gives the quotient formula. □

Formal correspondence. covered. The support algebra is constructed from the actual preorder. strictSupportIdeal is an actual TwoSidedIdeal of it, with membership proved equivalent to vanishing within every mutual-order class. Nonzero product blocks force strict paths; finite strict-path height is bounded and attained on nonempty indices. strictSupportIdeal_pow_eq_bot uses Mathlib standard ideal powers, with the span-of-words surrogate bridged by strictIdeal_power_mem. Antisymmetrization supplies the actual quotient partial order; class_maximalStrictSteps proves its height equals the exponent parameter. Grouped class projectors are actual sums of original projectors. The diagonal-class homomorphism is surjective with kernel equal to the strict ideal, and preorderQuotientDirectSumEquiv is the actual quotient by that ideal to the product of End of the literal direct sums of class packet ranges.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 847–875; proof lines 880–894.

Statement SHA-256. `89f93d5bdea5558d5ca9fef72742db6339af268623a37bedb7baabb777e4452a`

80. Transfer graph of an algebra-generating family

Stable ID. `PSC-I-D-TRANSFER-GRAPH`

Kind. definition

Statement.

Let $\mathcal{P} = (P_\alpha)_{\alpha \in I}$ be a finite packet system on X with every $P_\alpha \in \mathcal{A}$. Let $\mathcal{G} \subseteq \mathcal{A}$ satisfy

$$\mathcal{A} = \mathrm{alg}_{\Bbbk}(\{P_\alpha : \alpha \in I\} \cup \mathcal{G}),$$

where $\mathrm{alg}_{\Bbbk}(E)$ denotes the unital subalgebra over $\Bbbk$ generated by E. The transfer graph $\Gamma(\mathcal{G}, \mathcal{P})$ has vertex set I and an arrow

$$\alpha \longrightarrow \beta$$

when

$$P_\beta G P_\alpha \neq 0$$

for at least one $G \in \mathcal{G}$. Reachability includes paths of length zero.

Formal correspondence. definition represented. packetGeneratedAlgebra is Algebra.adjoin K (range P.proj ∪ ops), exactly the declared algebra in the manuscript. TransferEdge is witnessed by an actual member of ops with nonzero ambient transfer block. ReachableFrom uses ReflTransGen and hence includes length zero.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 916–931.

Statement SHA-256. `f6ddc4edec01afcf027a48300c34e66e9d1209d4f56bce17e3b9d00f88cd62cf`

81. Reachability support of the generated algebra

Stable ID. `PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT`

Kind. theorem

Statement.

For every $A \in \mathcal{A}$,

$$P_\beta A P_\alpha = 0 \qquad \text{whenever } \beta \notin \mathrm{Reach}(\alpha).$$

Equivalently,

$$\mathcal{A} \subseteq \mathfrak{M}_{R_\Gamma}, \qquad R_\Gamma := \{(\beta, \alpha) : \beta \in \mathrm{Reach}(\alpha)\}.$$

Proof. It is enough to treat a word in the generators and packet projectors. Insert

$$I = \sum_{\gamma \in I} P_\gamma$$

between consecutive nonprojector generators. Every resulting (β, α) block is a finite sum of products

$$P_\beta G_m P_{\gamma_{m-1}} G_{m-1} \cdots P_{\gamma_1} G_1 P_\alpha.$$

A nonzero product requires every consecutive block to be nonzero, hence gives a directed path

$$\alpha \to \gamma_1 \to \cdots \to \gamma_{m-1} \to \beta.$$

Packet projectors only insert stationary steps. Thus a word has no block from α to an unreachable β. Linear combinations preserve the same zero pattern. □

Formal correspondence. covered. Each projector and declared generator lies in the actual support algebra of ReflTransGen(TransferEdge). Algebra.adjoin_le therefore proves containment of the entire generated algebra. generatedAlgebra_block_zero is the literal ambient P_b A P_a=0 consequence outside reachability.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 934–947; proof lines 951–967.

Statement SHA-256. `7496f96565d405f8fe6b80491655dffe3b2ddad07b104c1131704464fb8ad031`

82. Reachability obstruction for generated hulls

Stable ID. `PSC-I-P-REACHABILITY-OBSTRUCTION`

Kind. proposition

Statement.

Let $I_0 \subseteq I$ and let

$$S \subseteq \bigoplus_{\alpha \in I_0} X_\alpha.$$

Then

$$\boxed{\mathrm{Hull}_{\mathcal{A}}(S) \subseteq \bigoplus_{\beta \in \mathrm{Reach}(I_0)} X_\beta.}$$

In particular, no seed supported outside a source strongly connected component can generate a nonzero vector in that source component.

Proof. For $s \in S$, write $s = \sum_{\alpha \in I_0} P_\alpha s$. If β is unreachable from I_0, then for every $A \in \mathcal{A}$,

$$P_\beta A s = \sum_{\alpha \in I_0} P_\beta A P_\alpha s = 0$$

by `PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT`. The same holds for every finite linear combination of orbit vectors. □

Formal correspondence. covered. The actual reachable-support subspace is preserved by every element of the generated algebra. Span of actual algebra orbits yields the hull inclusion for arbitrary seed sets. The source SCC is the actual mutual-reachability quotient, with source status equivalent to no incoming condensation edge. generatedAlgebra_source_obstruction separately assembles the printed final clause: seeds outside that source cannot generate any nonzero vector supported in it. It obtains a nonzero coordinate by packet completeness and applies no_reach_into_source and the unreachable-coordinate obstruction.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 977–992; proof lines 996–1005.

Statement SHA-256. `a46f6c292b4825e63689b6302077e4db2b2a8fe38f9958fc9018a623e7b8a239`

83. Packet-pure seed rank

Stable ID. `PSC-I-D-DIRECTED-SEED-RANK`

Kind. definition

Statement.

A vector is packet pure when it is zero or lies in one packet space X_α, equivalently when its packet support has cardinality at most one. Define

$$\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) := \min\{|S| : S \text{ is packet-pure and } \mathcal{A}S = X\}.$$

For $C \subseteq I$, put

$$P_C := \sum_{\alpha\in C} P_\alpha, \qquad X_C := P_C X, \qquad \mathcal{A}_C := P_C \mathcal{A} P_C,$$

and define the local packet pure rank

$$r_C := \operatorname{psrank}_{\mathcal{A}_C,\mathcal{P}|_C}(X_C).$$

Formal correspondence. definition represented. The cardinal infimum is over all actual pure generating subsets, its indexing type is nonempty, and its infimum is attained by well-ordering. Corner compression is e*T on range(e), hence literally e*T*e there. Closure and the identity restriction are proved. A local family retains outside indices as zero maps; local_proj_zero_iff identifies these precisely for nonzero original packets, and zero deletion preserves purity and rank. The corrected zero-or-range wording matches empty-index conventions.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 1011–1031.

Statement SHA-256. `2a2a8b2073162ddfce0cf7462765d5453176d369133cc86dc699c8d038e8fee8`

84. Basic packet-pure rank bounds

Stable ID. `PSC-I-P-PACKET-PURE-RANK-BOUNDS`

Kind. proposition

Statement.

For every A module resolved into packets with $P_\alpha \in A$,

$$\boxed{\operatorname{srank}_A(X) \le \operatorname{psrank}_{A,\mathcal{P}}(X) \le \sum_{\alpha\in I} \operatorname{srank}_{P_\alpha A P_\alpha}(X_\alpha).}$$

The corner algebra on the right is unital on X_α with identity P_α.

Proof. Every packet pure generating family is an unrestricted generating family, which gives the first inequality. For each α, choose a minimal generating family $S_\alpha \subseteq X_\alpha$ for the corner action $P_\alpha A P_\alpha$. The union $S = \bigcup_\alpha S_\alpha$ is packet pure. Since every $x \in X$ has the unique decomposition $x = \sum_\alpha x_\alpha$ and every x_α lies in $(P_\alpha A P_\alpha)S_\alpha \subseteq AS$, the union generates X. Its cardinality is at most the displayed sum. □

Formal correspondence. covered. The left inequality is direct inclusion of admissible generating sets. The upper inequality chooses attained minimal generating sets under each genuine corner algebra, embeds them into X, and proves their union generates by projector completeness and cor-

ner_action_ambient. The union cardinal is bounded by the cardinal sum. This works for arbitrary infinite dimensions and arbitrary cardinal ranks, including zero packets.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 1034–1048; proof lines 1052–1054.

Statement SHA-256. `11a3ad129e84e51edb000d4027f785874b59f3867639a7813ab9c8d944bf81c0`

85. Edge-surjective packet system

Stable ID. `PSC-I-D-EDGE-SURJECTIVE`

Kind. definition

Statement.

The module resolved into packets is surjective along edges relative to $\Gamma(\mathcal{G}, \mathcal{P})$ when, for every graph edge $\alpha \to \beta$, there exists $B \in \mathcal{A}$ such that the pure block

$$P_\beta B P_\alpha : X_\alpha \twoheadrightarrow X_\beta$$

is surjective.

Formal correspondence. definition represented. The edge is TransferEdge P ops, not a supplied surrogate graph. The witness T belongs to Algebra.adjoin of the projectors and declared operators; restrictBlocks is the literal map from the source range to the target range.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 1056–1063.

Statement SHA-256. `46dd6f245aadc93dd24ac5505cd37893d5474fb31208bab5babfacf0fa6d1ce4`

86. Source-component formula for packet-pure seed rank

Stable ID. `PSC-I-T-SOURCE-COMPONENT-SEED-RANK`

Kind. theorem

Statement.

Assume I is finite, every packet space is nonzero, and the module resolved into packets is surjective along edges. Let $\mathfrak{C}_0$ be the set of source strongly connected components of the condensation graph of $\Gamma(\mathcal{G}, \mathcal{P})$. Then

$$\boxed{\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) = \sum_{C \in \mathfrak{C}_0} r_C.}$$

Proof. Let S be a packet pure generating family for X. Discard zero seeds, which do not change the generated hull, and fix a source component C. Seeds outside C cannot reach C by `PSC-I-P-REACHABILITY-OBSTRUCTION`. Moreover, for $s \in X_C$ and $A \in \mathcal{A}$,

$$P_C A s = (P_C A P_C) s \in \mathcal{A}_C s.$$

Thus the seeds of S lying in X_C must generate X_C under $\mathcal{A}_C$, and there are at least r_C of them. Summing over disjoint source components gives

$$\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) \geq \sum_{C \in \mathfrak{C}_0} r_C.$$

For the reverse inequality, choose for each source component C a packet pure family S_C of size r_C generating X_C under $\mathcal{A}_C$, and put $S = \bigcup_C S_C$. Then every source component lies in $\mathcal{AS}$. Traverse the finite condensation graph in a topological order. Suppose all components preceding a component D that is not a source have already been shown to lie in $\mathcal{AS}$. Choose an incoming condensation edge from such a component to D, represented by a graph edge $\alpha \to \beta$ with α already generated and $\beta \in D$. Surjectivity along edges gives a pure block

$$B_{\beta\alpha} : X_\alpha \twoheadrightarrow X_\beta.$$

Because $X_\alpha \subseteq \mathcal{AS}$ and $\mathcal{AS}$ is invariant under $\mathcal{A}$, surjectivity implies $X_\beta \subseteq \mathcal{AS}$. For any other vertex $\gamma \in D$, choose a directed path inside the strongly connected component from β to γ. Successive surjective edge blocks propagate the full packet space along the path, so $X_\gamma \subseteq \mathcal{AS}$. Hence $X_D \subseteq \mathcal{AS}$. Induction over the condensation graph gives $X = \mathcal{AS}$, proving the upper bound. □

Formal correspondence. covered. SCC is the quotient by mutual reflexive-transitive reachability. Source is proved equivalent to absence of incoming condensation edges. Finite minimality proves every vertex is reached from a source. Generated-algebra support proves source projection compression. The lower bound removes zero seeds, proves those actually lying in each source generate its corner module, and injects the dependent union of these nonzero seed sets into the original generating set. The upper bound chooses attained local minima; actual surjective blocks propagate range containment along actual paths. No source coverage, compression identity, generating conclusion, or graph certificate is assumed in the final theorem. The theorem in fact does not require nonzero original packets, so it includes the printed hypotheses.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 1066–1076; proof lines 1081–1098.

Statement SHA-256. `0be97dd456719faad7e8e6ba4091ee877f58e7c7c900c8ae79a7b8385c19daea`

87. Packet-propagating system

Stable ID. `PSC-I-D-PACKET-PROPAGATING`

Kind. definition

Statement.

A packet system that is surjective along edges is called packet propagating when every packet space X_α is cyclic under its diagonal corner algebra

$$P_\alpha \mathcal{A} P_\alpha.$$

Formal correspondence. definition represented. Cyclicity is the existing module Cyclic predicate for the genuine packetCornerAlgebra acting on its actual range. There are no extra local-rank-one certificates supplied.

Source locator. `editable/traditional/03_packets_and_graphs.tex`; statement lines 1105–1111.

Statement SHA-256. `4e9bfc875f14eeec60ebfab1d4906a00230861dfe0e37ea46c90639653fbc111`

88. Unweighted source-component count

Stable ID. PSC-I-C-SOURCE-COMPONENT-COUNT

Kind. corollary

Statement.

Assume the module resolved into packets is packet propagating and $X_\alpha \neq 0$ for every $\alpha \in I$. If c_0 denotes the number of source strongly connected components, then

$$\boxed{\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) = c_0.}$$

Proof. Fix a source component C and a vertex $\alpha \in C$. Choose a cyclic vector $s \in X_\alpha$ for $P_\alpha \mathcal{A} P_\alpha$. It generates all of X_α. Strong connectivity and surjectivity along edges then propagate the whole packet space to every vertex of C. Thus $r_C = 1$ for every source component. Apply PSC-I-T-SOURCE-COMPONENT-SEED-RANK. □

Formal correspondence. covered. The upper bound chooses a cyclic vector at one actual representative vertex of each source SCC and propagates full packet ranges to every vertex. The lower bound proves every generating set has a nonzero source projection, then uses purity and orthogonality to inject Sources into the chosen pure generating set. Nonzero packet range hypotheses are explicit. The finite cardinal-number form is proved; no finite generation or dimension assumption is introduced.

Source locator. editable/traditional/03_packets_and_graphs.tex; statement lines 1114–1122; proof lines 1126–1128.

Statement SHA-256. f7d07d0e6a080b4b92c8a401d9663246fb377ecc6d930b8ecdb94ec07ea7e5dc

89. Mixed-support seeds can merge source requirements

Stable ID. PSC-I-B-PACKET-PURE-BOUNDARY

Kind. boundary

Statement.

The source component formulas concern packet pure seeds. They do not give the unrestricted seed rank. For example, let

$$\mathcal{A} = \Bbbk \times \Bbbk, \qquad X = \Bbbk \oplus \Bbbk$$

with the coordinate action and the two coordinate packet projectors. The transfer graph has two source components and

$$\operatorname{psrank}_{\mathcal{A},\mathcal{P}}(X) = 2,$$

but the mixed vector $(1, 1)$ is cyclic, so

$$\operatorname{srank}_{\mathcal{A}}(X) = 1.$$

Formal correspondence. scope boundary. The unrestricted mixed-seed example is consistent with the proved product-algebra formula. The packet-pure rank calculation is not separately formalized.

Source locator. editable/traditional/03_packets_and_graphs.tex; statement lines 1130–1146.

Statement SHA-256. a3610fb8adcedb4fb91006a77f357f3f18c57678d044c21b8522730a456b0006

A.3 Transport and realization

90. Coordinate transport

Stable ID. `PSC-I-T-COORDINATE-TRANSPORT`

Kind. theorem

Statement.

Let $U : X \to Y$ be a vector space isomorphism and let $\mathcal{P} = (P_\alpha)$ be a packet system on X. Define

$$Q_\alpha := UP_\alpha U^{-1}.$$

Then $\mathcal{Q} = (Q_\alpha)$ is a packet system on Y. For $\widetilde{T} = UTU^{-1}$,

$$\boxed{Q_\beta \widetilde{T} Q_\alpha = U(P_\beta T P_\alpha)U^{-1}.}$$

Hence packet dimensions, zero patterns, and support preorders are invariant under coordinate transport. If a declared algebra generating family $\mathcal{G}$ is transported to $U\mathcal{G}U^{-1}$, then its transfer graph and reachability relation are also preserved.

Proof. Conjugation preserves products and sums, so

$$Q_\alpha Q_\beta = UP_\alpha P_\beta U^{-1} = \delta_{\alpha\beta} Q_\alpha, \qquad \sum_\alpha Q_\alpha = I_Y.$$

The block identity is direct multiplication:

$$Q_\beta \widetilde{T} Q_\alpha = UP_\beta T P_\alpha U^{-1}.$$

All listed invariants are determined by these transported blocks. □

Formal correspondence. covered. conjugate constructs the actual family U P_i U^{-1} and proves all projector identities. conjugate_transferBlock proves the exact displayed equality. packetRangeEquiv restricts U to each actual packet range and yields cardinal dimension equality with explicit universe lifts. support_pattern_invariant proves preservation for every relation. TransferEdge and ReachableFrom are unchanged when the actual generator set is transported by conjugation.

Source locator. `editable/traditional/04_transport.tex`; statement lines 7–22; proof lines 26–41.

Statement SHA-256. `ae5e550ba65822235a1f8ab84c76a5a3ac876b28334523c9d845ece2af2d2d5a`

91. Packet-compatible operator morphism

Stable ID. `PSC-I-D-PACKET-MORPHISM`

Kind. definition

Statement.

Let $(X, \mathcal{A}, \mathcal{P})$ and $(Y, \mathcal{B}, \mathcal{Q})$ be packet resolved operator modules with packet index sets I and J. A packet compatible operator morphism is a triple

$$(\Phi, \varphi, \sigma)$$

consisting of a unital algebra homomorphism $\varphi : \mathcal{A} \to \mathcal{B}$, a linear map $\Phi : X \to Y$ satisfying

$$\Phi(Ax) = \varphi(A)\Phi(x),$$

and a map $\sigma : I \to J$ satisfying

$$Q_j \Phi = \Phi P_{\sigma^{-1}(j)}, \qquad P_{\sigma^{-1}(j)} := \sum_{\sigma(\alpha)=j} P_\alpha.$$

An empty fiber is interpreted as the zero projector.

Formal correspondence. definition represented. The source and target have actual concrete unital operator subalgebras and nonzero complete orthogonal packet projectors. The linear map, unital algebra map and entirely arbitrary index map are separately retained. Fiber projectors are literal finite sums; no injectivity or surjectivity condition is imposed on the index map.

Source locator. `editable/traditional/04_transport.tex`; statement lines 48–68.

Statement SHA-256. `e0e522a401795ef9ecf2497c058baf522801bf33dd581a837deaabd9cbe887bf`

92. Morphism of seeded systems

Stable ID. `PSC-I-D-SEED-MORPHISM`

Kind. definition

Statement.

A morphism

$$(X, \mathcal{A}, S, \mathcal{P}) \longrightarrow (Y, \mathcal{B}, S', \mathcal{Q})$$

is a packet compatible operator morphism satisfying the seed condition

$$\Phi(S) \subseteq \mathrm{Hull}_{\mathcal{B}}(S').$$

In the topological category all maps and actions are required to be continuous and the seed condition uses the topological hull when declared.

Formal correspondence. definition represented. The algebraic condition uses the actual target algebra span, with arbitrary seed sets. The continuous variant separately retains Hausdorff topological vector-space structure, actual continuity of every algebra operator and packet projector, and actual continuity of the carrier morphism. Its seed condition is membership in the literal topologicalClosure of AlgebraGeneratedHull. The paper specifies an algebra of continuous operators, not a separate topology on that algebra, and the code states this convention explicitly.

Source locator. `editable/traditional/04_transport.tex`; statement lines 73–84.

Statement SHA-256. `88c7e9a865f68bac1df1727bd7cfc8c66e5b11d2e8be77ada048a83aaafd5637`

93. Naturality of generated hulls and transfer blocks

Stable ID. `PSC-I-T-NATURALITY`

Kind. theorem

Statement.

Let (Φ, φ, σ) be a packet compatible operator morphism. For every subset $S \subseteq X$,

$$\boxed{\Phi(\mathrm{Hull}_{\mathcal{A}}(S)) = \mathrm{Hull}_{\varphi(\mathcal{A})}(\Phi(S)) \subseteq \mathrm{Hull}_{\mathcal{B}}(\Phi(S)).}$$

For $T \in \mathcal{A}$ and $i, j \in J$,

$$\boxed{Q_j \varphi(T) Q_i \Phi = \sum_{\substack{\sigma(\beta)=j \\ \sigma(\alpha)=i}} \Phi P_\beta T P_\alpha.}$$

If Φ is surjective and S generates X, then $\Phi(S)$ generates Y under $\varphi(\mathcal{A})$ and hence under $\mathcal{B}$. If, in addition, the morphism is seeded, then S' generates Y.

Proof. Every element of $\Phi(\mathrm{Hull}_{\mathcal{A}}(S))$ is a finite sum of vectors

$$\Phi(As) = \varphi(A)\Phi(s),$$

so it belongs to $\mathrm{Hull}_{\varphi(\mathcal{A})}(\Phi(S))$. Conversely, every generator $\varphi(A)\Phi(s)$ equals $\Phi(As)$, giving equality.

For the block formula, use intertwining and packet compatibility twice:

$$\begin{aligned} Q_j \varphi(T) Q_i \Phi &= Q_j \varphi(T) \Phi P_{\sigma^{-1}(i)} \\ &= Q_j \Phi T P_{\sigma^{-1}(i)} \\ &= \Phi P_{\sigma^{-1}(j)} T P_{\sigma^{-1}(i)} \\ &= \sum_{\substack{\sigma(\beta)=j \\ \sigma(\alpha)=i}} \Phi P_\beta T P_\alpha. \end{aligned}$$

If Φ is surjective and $\mathrm{Hull}_{\mathcal{A}}(S) = X$, the hull equality gives $Y = \mathrm{Hull}_{\varphi(\mathcal{A})}(\Phi(S))$. If $\Phi(S) \subseteq \mathrm{Hull}_{\mathcal{B}}(S')$, then $\mathrm{Hull}_{\mathcal{B}}(\Phi(S)) \subseteq \mathrm{Hull}_{\mathcal{B}}(S')$, so S' generates Y. □

Formal correspondence. covered. Hull naturality is actual Submodule.map equality with the span of image-algebra orbits; target-algebra inclusion is derived. The block formula is the exact double sum over possibly many-to-one fibers, at the level of actual linear maps. Surjective generation is derived from the map of top, then the seeded condition implies generation by the target seeds.

Source locator. `editable/traditional/04_transport.tex`; statement lines 87–109; proof lines 114–131.

Statement SHA-256. `e7fb174db6031054bdf06996ebad915dfaf307ad38b27fceb17442c46128863b`

94. Category of seeded packet systems

Stable ID. `PSC-I-P-SEED-SYSTEM-CATEGORY`

Kind. proposition

Statement.

Packet compatible operator morphisms compose, seeded morphisms compose, and identity morphisms satisfy both definitions. Thus packet resolved operator modules form a category **PktMod** and seeded packet systems form a category **SeedSys**.

Proof. Let $(\Phi, \varphi, \sigma) : (X, \mathcal{A}, \mathcal{P}) \to (Y, \mathcal{B}, \mathcal{Q})$ and $(\Psi, \psi, \tau) : (Y, \mathcal{B}, \mathcal{Q}) \to (Z, \mathcal{C}, \mathcal{R})$ be morphisms. Intertwining is preserved because

$$\Psi\Phi(Ax) = \Psi(\varphi(A)\Phi x) = (\psi \circ \varphi)(A)\Psi\Phi x.$$

For k in the target packet set,

$$\begin{aligned} R_k\Psi\Phi &= \Psi Q_{\tau^{-1}(k)}\Phi \\ &= \Psi\Phi \sum_{\tau(\sigma(\alpha))=k} P_\alpha. \end{aligned}$$

Thus the composite packet map is $\tau \circ \sigma$. If the morphisms are seeded, then

$$\Phi(S) \subseteq \mathrm{Hull}_{\mathcal{B}}(S'), \qquad \Psi(S') \subseteq \mathrm{Hull}_{\mathcal{C}}(S'').$$

By `PSC-I-T-NATURALITY`,

$$\Psi\Phi(S) \subseteq \Psi(\mathrm{Hull}_{\mathcal{B}}(S')) \subseteq \mathrm{Hull}_{\mathcal{C}}(\Psi(S')) \subseteq \mathrm{Hull}_{\mathcal{C}}(S'').$$

Identity maps satisfy the definitions with identity algebra and packet maps. □

Formal correspondence. covered. Composition explicitly aggregates composite fibers by a finite-sum rearrangement and composes operator intertwiners. Algebraic seed closure is proved using span transport. The continuous variant proves invariance of the actual closed algebraic hull from individual operator continuity, maps the source algebraic hull into the target closed hull, then derives map_closedHull from carrier continuity and closure minimality. Seeded composition applies this proved closure transport; identity seed membership and all category laws are proved. The actual algebraic and topological Category instances therefore cover the full stated category claim.

Source locator. `editable/traditional/04_transport.tex`; statement lines 138–141; proof lines 147–180.

Statement SHA-256. `3cc5c23c5df9094cc9c18b7b9539bf0cadbbe2e74dd249651920fd5034f347a1`

95. Packet exact sequence for an invariant subspace

Stable ID. `PSC-I-T-PACKET-EXACT-SEQUENCE`

Kind. theorem

Statement.

Let $K \leq X$ satisfy $P_\alpha K \subseteq K$ for every α. Define

$$K_\alpha := P_\alpha K = K \cap X_\alpha$$

and

$$\overline{P}_\alpha(x + K) := P_\alpha x + K.$$

Then:

(i) The nonzero restrictions $P_\alpha|_K$ form a packet system on K.

(ii) The nonzero $\overline{P}_\alpha$ form a packet system on X/K.

(iii) For every α there is a short exact sequence

$$0 \longrightarrow K_\alpha \longrightarrow X_\alpha \longrightarrow \overline{P}_\alpha(X/K) \longrightarrow 0,$$

and

$$\overline{P}_\alpha(X/K) \cong X_\alpha/(X_\alpha \cap K) \cong (X_\alpha + K)/K.$$

(iv) The full exact sequence decomposes packetwise as the direct sum of these short exact sequences.

Proof. Restriction preserves idempotency, pairwise annihilation, and the identity sum on K. If $x \in K \cap X_\alpha$, then $x = P_\alpha x \in P_\alpha K$, proving $K_\alpha = K \cap X_\alpha$.

The quotient operator is well defined because replacing x by $x + k$ changes $P_\alpha x$ by $P_\alpha k \in K$. The idempotent and annihilation identities descend, and the sum is the identity on X/K. The map $X_\alpha \to X/K$ has image $(X_\alpha + K)/K$ and kernel $X_\alpha \cap K$, giving the short exact sequence and the quotient isomorphisms. Exact packet coordinates identify the original sequence with the direct sum over α. □

Formal correspondence. covered. K is an actual invariant submodule. Restriction/quotient projectors are constructed; restricted range equals K intersect X_i. Actual packet inclusion is injective, its range equals the actual quotient-map kernel, and quotient map is onto. First/second quotient isomorphisms use denominator comaps in the correct ambient type. Pointwise coordinate maps intertwine with original inclusion/quotient and form a short exact sequence; exact direct-sum coordinates give the full decomposition.

Source locator. `editable/traditional/04_transport.tex`; statement lines 185–216; proof lines 221–225.

Statement SHA-256. `7311937b07e98adf3e6f91ad665358380cab6ce26e3daeffdbf4787d7a01de56`

96. Packet descent through an invariant quotient

Stable ID. PSC-I-T-QUOTIENT-PACKET-DESCENT

Kind. theorem

Statement.

Under the hypotheses of PSC-I-T-PACKET-EXACT-SEQUENCE, the operators

$$\overline{P}_\alpha(x+K) = P_\alpha x + K$$

are well defined pairwise annihilating idempotents with sum $I_{X/K}$, and the quotient map satisfies

$$\boxed{\pi P_\alpha = \overline{P}_\alpha \pi.}$$

After zero packets are removed, they form the canonical quotient packet system.

Proof. All assertions are the quotient part of PSC-I-T-PACKET-EXACT-SEQUENCE. The intertwining identity follows directly from the definition:

$$\overline{P}_\alpha \pi(x) = P_\alpha x + K = \pi P_\alpha(x).$$

□

Formal correspondence. covered. Actual mapQ uses invariance of K. Quotient representative calculations prove the laws and commuting identity. Zero removal preserves completeness. Quotient surjectivity proves uniqueness of every descended operator, so canonicity is established.

Source locator. `editable/traditional/04_transport.tex`; statement lines 232–243; proof lines 247–252.

Statement SHA-256. `b51391a00350d220fc92610f351826489ca69d166e27e266a1ecc6ecc7a2ffc7`

97. Restriction and quotient of transfer blocks

Stable ID. PSC-I-C-QUOTIENT-PACKET-DESCENT

Kind. corollary

Statement.

Under the hypotheses of PSC-I-T-PACKET-EXACT-SEQUENCE, let $T \in \mathrm{End}(X)$ preserve K. Then every block $T_{\beta\alpha}$ maps K_α into K_β, and the induced quotient operator $\overline{T}$ satisfies

$$\boxed{\overline{T}_{\beta\alpha}\,\pi = \pi T_{\beta\alpha},}$$

where $\pi : X \twoheadrightarrow X/K$.

Proof. If $x \in K_\alpha$, then $Tx \in K$, so

$$T_{\beta\alpha}x = P_\beta T x \in K \cap X_\beta = K_\beta.$$

On the quotient,

$$\overline{T}_{\beta\alpha}\pi = \overline{P}_\beta \overline{T P}_\alpha \pi = \pi P_\beta T P_\alpha.$$

□

Formal correspondence. covered. Invariance under P_i,T,P_j proves preservation of K and membership in the target packet. Tbar is actual K.mapQ K T hT, and representative calculation proves the quotient-block identity. Lean orientation is the symmetric equality of the printed formula.

Source locator. `editable/traditional/04_transport.tex`; statement lines 254–265; proof lines 269–282.

Statement SHA-256. `4fa0ae8effe1e5472dcde2be67829cba4ae586df9853d252baff055f3ee7fd46`

98. Topological packet exact sequence

Stable ID. `PSC-I-C-TOPOLOGICAL-PACKET-EXACT-SEQUENCE`

Kind. corollary

Statement.

If X is Banach, the P_α are bounded, and K is closed and packet invariant, then every packet sequence in `PSC-I-T-PACKET-EXACT-SEQUENCE` is a short exact sequence of Banach spaces with bounded maps, and the quotient projectors are bounded.

Proof. Closedness of K makes X/K Banach. The restricted projectors are bounded, and the quotient projectors are bounded because

$$\|\overline{P}_\alpha(x+K)\|_{X/K} \le \|P_\alpha\|\,\|x+K\|_{X/K}.$$

The packet spaces are closed ranges of bounded idempotents. □

Formal correspondence. covered. The generic theorem works over any NormedField, including a trivial valuation, with explicit boundedness of the original projectors. Restriction and quotient CLMs are the actual algebraic packet maps. Near-minimal quotient representatives transfer every original bound to the quotient; the norm bound uses the usual infimum-of-bounds norm of the bounded additive projector. Actual packet inclusion and quotient maps are bounded with constant one and retain injectivity, exactness, and surjectivity. Closed K and closed bounded-idempotent ranges make all three terms complete.

Source locator. `editable/traditional/04_transport.tex`; statement lines 284–287; proof lines 292–300.

Statement SHA-256. `2590b3fce057e3d054522a58a728f46dbf3c609c95ff6ab061ff654ffd72fee1`

99. Dense realization theorem

Stable ID. `PSC-I-T-DENSE-REALIZATION`

Kind. theorem

Statement.

Let X and Y be Hausdorff topological vector spaces, let $\Phi : X \to Y$ be continuous, surjective, and intertwining with respect to φ, and let S topologically generate X. Then $\Phi(S)$ topologically generates Y. If finite continuous packet systems satisfy

$$Q_j \Phi = \Phi P_{\sigma^{-1}(j)},$$

then

$$\boxed{Q_j Y = \Phi(P_{\sigma^{-1}(j)} X)}$$

for every j.

Proof. Given $y = \Phi(x)$, choose a net $x_\lambda \in \mathrm{Hull}_{\mathcal{A}}(S)$ converging to x. Then

$$\Phi(x_\lambda) \to y,$$

and naturality places every $\Phi(x_\lambda)$ in $\mathrm{Hull}_{\mathcal{B}}(\Phi(S))$. Hence the latter hull is dense. For packets, one inclusion follows from compatibility. Conversely, if $y = Q_j y$, choose x with $\Phi(x) = y$. Then

$$y = Q_j \Phi(x) = \Phi P_{\sigma^{-1}(j)} x.$$

□

Formal correspondence. covered. For the actual continuous linear intertwiner in the packet-morphism convention, the image of the source algebraic operator hull is contained in the target hull, and continuous surjectivity transports density. The actual finite grouped projector is the sum over the σ-fibre, including empty fibres. Surjectivity and the given intertwining identity prove exact equality of image and target range. No norm or metrizability assumptions occur.

Source locator. `editable/traditional/04_transport.tex`; statement lines 305–318; proof lines 322–331.

Statement SHA-256. `58bf2b9bfb4ab3ae9f3309b03cc231e0b59c81b7fe2b6c99e4b74884aaf15bca`

100. Quotient topology and completed realization

Stable ID. `PSC-I-T-QUOTIENT-TOPOLOGY-COMPLETION`

Kind. theorem

Statement.

Let $\Phi : X \twoheadrightarrow Y$ be a continuous linear quotient map between Hausdorff topological vector spaces, and put $K = \operatorname{Ker} \Phi$. Then the induced map

$$\widetilde{\Phi} : X/K \longrightarrow Y$$

is a topological isomorphism. If $D \subseteq X$ is a linear subspace whose image $\Phi(D)$ is dense in a complete space Y, then the completion of

$$D/(D \cap K)$$

in the topology transported from Y is canonically isomorphic to Y.

Proof. The algebraic first isomorphism theorem gives a bijection $\widetilde{\Phi}$. By definition of the quotient topology, a set $U \subseteq X/K$ is open exactly when its preimage in X is open. Because Φ is a quotient map, this is equivalent to $\widetilde{\Phi}(U)$ being open in Y. Thus $\widetilde{\Phi}$ is a homeomorphism. The second statement applies the dense completion argument of PSC-I-T-COMPLETED-PRESENTATION to the injective map $D/(D \cap K) \to Y$. □

Formal correspondence. covered. The actual first-isomorphism linear equivalence is continuous both ways using IsQuotientMap, not continuity plus surjectivity alone. The restriction to D uses its actual kernel, definitionally comap(D inclusion,ker Φ); density is exactly Φ(D). The same explicit transported-uniformity quotient completion gives the canonical target equivalence and agrees with Φ on represented classes. No original-domain quotient topology is silently substituted for the printed transported topology.

Source locator. `editable/traditional/04_transport.tex`; statement lines 333–344; proof lines 348–350.

Statement SHA-256. `2e986d76e93d67446708502bfae8275f63d8ab0eaf8dcc3e3630ee7f6b3fa077`

101. Continuous surjections and quotient topology

Stable ID. PSC-I-B-TOPOLOGICAL-QUOTIENT

Kind. boundary

Statement.

Continuity and surjectivity transport density as in PSC-I-T-DENSE-REALIZATION. They do not by themselves identify the given topology on Y with the quotient topology from X. The stronger conclusion in PSC-I-T-QUOTIENT-TOPOLOGY-COMPLETION requires the quotient map hypothesis or an equivalent open mapping condition.

Formal correspondence. scope boundary. The quotient-map requirement is preserved; no open-map theorem is inferred from continuity alone.

Source locator. `editable/traditional/04_transport.tex`; statement lines 352–355.

Statement SHA-256. `855989a0fadfaf7174d83889d9a2c4208bc722f866b35c5da99de3e7ecf1b9f7`

102. Realization functor and preservation profile

Stable ID. `PSC-I-D-REALIZATION-FUNCTOR`

Kind. definition

Statement.

Let $\mathbf{SeedSys}_{\mathrm{abs}}$ be an abstract category of seed systems and let $\mathbf{C}_D$ be a domain category. A realization functor is a functor

$$\mathsf{Ev}_D : \mathbf{SeedSys}_{\mathrm{abs}} \longrightarrow \mathbf{C}_D$$

with a declared preservation profile specifying which of the following structures are preserved:

$$0,\ I,\ \text{finite sums},\ \text{composition},\ \text{biproducts},$$
$$\text{kernels},\ \text{cokernels},\ \text{quotients},\ \text{tensor products},$$
$$\text{packet idempotents},\ \text{transfer blocks},\ \text{topological completions}.$$

Preservation of a structure means preservation of its defining identity or universal property, not merely a similarity of notation.

Formal correspondence. documentary. The profile is represented by actual additive/linear functor and PreservesLimit/PreservesColimit instances, with typed syntax for equational operations. Universal constructions also use explicit structured categories, an induced functor, an underlying comparison isomorphism, and actual initial/terminal preservation. No Boolean profile field is accepted in place of preservation evidence.

Source locator. `editable/traditional/04_transport.tex`; statement lines 382–397.

Statement SHA-256. `152d8ee6b5bb1735f7655c6eeee72210d213959c00c9e6168d554eee25b7c5fe`

103. Equational realization principle

Stable ID. `PSC-I-T-REALIZATION-EXPORT`

Kind. theorem

Statement.

Let $F : \mathbf{C} \to \mathbf{D}$ be an additive functor between preadditive categories. Any equality of morphisms constructed from declared generating morphisms by finite sums, additive inverses, identities, zero maps, and composition is preserved by F. Scalar multiplication by a coefficient ring is also preserved when the categories are enriched over that ring and F is linear for the declared enrichment. If a proof additionally uses a chosen biproduct, kernel, cokernel, quotient, tensor product, or completion and F preserves that construction in its declared profile, then the corresponding proof diagram is transported to $\mathbf{D}$.

Proof. Functoriality gives

$$F(I) = I, \qquad F(g \circ f) = F(g) \circ F(f),$$

and additivity gives

$$F(0) = 0, \qquad F(f+g) = F(f) + F(g), \qquad F(-f) = -F(f).$$

If the declared enrichment is present, linearity additionally gives $F(af) = aF(f)$ for every coefficient a. A structural induction on the construction of a morphism expression therefore carries every permitted

equational proof to its image under F. If a step invokes a universal construction, preservation means that the image diagram satisfies the same universal property. The uniquely induced morphism in the source proof is therefore carried to the uniquely induced morphism in the target proof. Repeating this argument through the finite proof diagram transports the conclusion. □

Formal correspondence. covered. Structural induction on well-typed morphism expressions proves transport of identities, zero, addition, negation, scalar multiplication and composition. The additive-only statement uses the canonical integer enrichment, and no extra scalar-linearity assumption is inserted. Actual limit/colimit diagrams are transported under the matching preservation instance. The conditional universal-construction clause is explicit: a declared profile supplies the structured categories of the universal problem, an induced functor, compatibility with the underlying realization functor, and actual initial/terminal preservation. CategoryUniversal derives the mapped initial/terminal property, equality of the uniquely induced arrows, and the underlying commuting comparison squares. Tensor and completion preservation are legitimate conditional inputs, not falsely inferred from additivity.

Source locator. `editable/traditional/04_transport.tex`; statement lines 400–403; proof lines 408–424.

Statement SHA-256. `e0352bdb7ba75e24d4c7e6e0259802f47800c4945c13efe4b3c68edcacef678d`

104. Realization export principle

Stable ID. `PSC-I-P-REALIZATION-EXPORT`

Kind. proposition

Statement.

Every canonical Paper I identity whose proof uses only the operations listed in a realization functor's verified preservation profile remains valid after applying that functor. If the proof uses a domain specific hypothesis not encoded by the functor, that hypothesis must be verified separately before the transported statement is promoted to a theorem.

Proof. The first assertion is `PSC-I-T-REALIZATION-EXPORT`. The second is a logical boundary: a functor transports established identities and preserved universal constructions, but it does not create an unverified analytic, topological, positivity, finiteness, or integrality hypothesis. □

Formal correspondence. covered. This is the methodological specialization of record 103 to already established Paper I identities. The formal equations are transported from a proved source equality, not assumed as the desired target equality. The universal-property step uses the explicit structured-category profile of CategoryUniversal. No new analytic/topological/positivity/finiteness/integrality assumption is generated.

Source locator. `editable/traditional/04_transport.tex`; statement lines 432–435; proof lines 439–441.

Statement SHA-256. `bf39b86447c8332d60216e48d23129b6ae5b096611e3febd9d09a699f1a9a557`

105. Semantic realization criterion

Stable ID. `PSC-I-D-SEMANTIC-REALIZATION`

Kind. definition

Statement.

A domain theorem is a semantic realization of a canonical seed calculus theorem only when:

(i) An explicit realization functor or intertwining construction is given.

(ii) The preservation profile covers every construction used in the canonical proof.

(iii) Every additional analytic, finiteness, convergence, positivity, or integrality hypothesis is verified in the domain.

(iv) The resulting conclusion is stated in the domain's native objects.

Reuse of the words "seed", "packet", or "transfer" without these data is only syntactic reuse.

Formal correspondence. documentary. A methodology and scope checklist, not automatic formal certification.

Source locator. `editable/traditional/04_transport.tex`; statement lines 443–453.

Statement SHA-256. `7bfb521bf39d1f519716ff37b9ffcf3ae76ce4f62fe121586daa29e66e7223df`

106. Reserved example slots

Stable ID. `PSC-I-ALG-EXAMPLE-SLOTS`

Kind. definition

Statement.

The foundational calculus is illustrated by four canonical algebraic realizations:

(i) A typed quiver reachability model.

(ii) An algebraic coordinate ring presentation.

(iii) A companion and repeated primary single operator model.

(iv) Path and cycle adjacency models.

The worked model sections below instantiate these four interfaces. A bilateral shift example records the analytic completion mechanism also used in Paper V. Computational specifications connect the proofs with the verification companion and are not premises of the foundational theorems.

Formal correspondence. documentary. Example slots are editorial. Existing examples and algorithm specifications remain outside the 69 proved-record denominator.

Source locator. `editable/traditional/04_transport.tex`; statement lines 470–480.

Statement SHA-256. `a870d22072364c0a743aede20db499ccda1ce4c62a009a82dbaefd76f525375d`

107. Paper I export object

Stable ID. PSC-I-EXPORT

Kind. definition

Statement.

The canonical Paper I export is

$$\mathsf{PSC}_I = \left(\begin{array}{c} \text{typed grammars and realized operator algebras,} \\ \text{seed systems, free presentations, and seed rank,} \\ \text{module-top, semisimple, and single-operator formulas,} \\ \text{finite and unconditional packet coordinates,} \\ \text{transfer blocks and support algebras,} \\ \text{reachability and packet-pure source generation,} \\ \text{morphisms, invariant exact sequences, quotients, completions,} \\ \text{realization functors and equational theorem transport} \end{array} \right).$$

Formal correspondence. documentary. An export catalogue, not one additional kernel theorem or a completed formal package object.

Source locator. editable/traditional/04_transport.tex; statement lines 483–503.

Statement SHA-256. 5c58d6e198263b11ae3e1f98701de3cf1b70b5bc5da229d1ed705b607efe2aa7

108. Exact limits of Paper I

Stable ID. PSC-I-B-FINAL-CLAIM-BOUNDARY

Kind. boundary

Statement.

The existence of an exact packet decomposition does not imply that the decomposition is computationally discoverable, numerically well conditioned, statistically identifiable, dynamically invariant, or mathematically useful. The condition number in PSC-I-D-PACKET-CONDITION-NUMBER quantifies only coordinate stability once the packets are supplied. Refinement towers, filtered transfer, nonlinear obstruction objects, automatic packet recovery, continuum limits, arithmetic lifts, and automorphic occupancy require additional theorems in later papers.

Formal correspondence. scope boundary. No discoverability, conditioning, identifiability or later-paper existence claim follows automatically from finite packet identities.

Source locator. editable/traditional/04_transport.tex; statement lines 532–535.

Statement SHA-256. d059c43a34200d583173f81a335b2acb713d6ddbf0d46054763a575b1fdef0c7

B Typed core signatures

These twenty signatures are reproduced from `kir/core_typed_signatures.json`. Unicode mathematical characters are represented with standard JSON escapes in the listings. Their meanings are unchanged. The schema does not make these strings into independently checked proof terms.

Linear seed system

PSC-I-D-SEED-SYSTEM

```
{
  "params": [
    "A:OperatorAlgebra",
    "M:Module(A)",
    "S:FiniteFamily(M)"
  ],
  "returns": "SeedSystem(A,M,S)"
}
```

Generated hull

PSC-I-D-GENERATED-HULL

```
{
  "params": [
    "A:OperatorAlgebra",
    "M:Module(A)",
    "S:Family(M)"
  ],
  "returns": "Submodule(M)"
}
```

Seed rank

PSC-I-D-SEED-RANK

```
{
  "params": [
    "A:OperatorAlgebra",
    "M:Module(A)"
  ],
  "returns": "Cardinal"
}
```

Finite packet system

PSC-I-D-PACKET-SYSTEM

```
{
  "params": [
    "K:Field",
    "X:VectorSpace(K)",
    "I:FiniteIndexSet",
    "P:I\u2192End(X)"
  ],
  "requires": [
    "idempotent(P)",
    "pairwise_annihilating(P)",
```

```
    "sum(P)=id_X"
  ],
  "returns": "PacketSystem(X,I)"
}
```

Packet analysis and synthesis

PSC-I-D-ANALYSIS-SYNTHESIS

```
{
  "params": [
    "P:PacketSystem(X,I)"
  ],
  "returns": [
    "Analyze_P:X\u2192DirectSum(I,PacketSpace(P,i))",
    "Synthesize_P:DirectSum(I,PacketSpace(P,i))\u2192X"
  ]
}
```

Exact packet-coordinate isomorphism

PSC-I-T-EXACT-PACKET-COORDINATES

```
{
  "params": [
    "P:PacketSystem(X,I)"
  ],
  "returns": "Equivalence(X,DirectSum(I,PacketSpace(P,i)))",
  "certificate": "exact_inverse"
}
```

Transfer block

PSC-I-D-TRANSFER-BLOCK

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "T:End(X)",
    "alpha:I",
    "beta:I"
  ],
  "returns": "LinearMap(PacketSpace(P,alpha),PacketSpace(P,beta))"
}
```

Exact operator reconstruction

PSC-I-T-OPERATOR-RECONSTRUCTION

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "T:End(X)"
  ],
  "returns": "Equality(T,Sum(alpha,beta,TransferBlock(P,T,beta,alpha)))"
}
```

Packet matrix multiplication

PSC-I-T-BLOCK-MULTIPLICATION


```
{
  "params": [
    "P:PacketSystem(X,I)",
    "S:End(X)",
    "T:End(X)",
    "alpha:I",
    "gamma:I"
  ],
  "returns": "Equality(Block(S\u2218T,gamma,alpha),Sum(beta,Block(S,gamma,beta)\u2218Block(T,beta,alpha)))"
}
```


Packet powers and paths

PSC-I-C-BLOCK-POWERS


```
{
  "params": [
    "P:PacketSystem(X,I)",
    "T:End(X)",
    "m:Nat"
  ],
  "returns": "PacketPathExpansion(T,m)"
}
```


Blockwise intertwining equations

PSC-I-T-BLOCK-INTERTWINING


```
{
  "params": [
    "P:PacketSystem(X,I)",
    "Q:PacketSystem(Y,J)",
    "T:End(X)",
    "S:End(Y)",
    "U:LinearMap(X,Y)"
  ],
  "returns": "Iff(U\u2218T=S\u2218U,LocalBlockIntertwining(P,Q,T,S,U))"
}
```


Full block-algebra representation

PSC-I-T-BLOCK-ALGEBRA


```
{
  "params": [
    "P:PacketSystem(X,I)"
  ],
  "returns": "AlgebraEquiv(End(X),DependentBlockAlgebra(P))"
}
```

Support relation and support space

PSC-I-D-SUPPORT-RELATION

```
{
  "params": [
    "I:FiniteIndexSet",
    "R:Relation(I)"
  ],
  "returns": "SupportSpace(R)"
}
```

Coordinate transport

PSC-I-T-COORDINATE-TRANSPORT

```
{
  "params": [
    "U:LinearEquiv(X,Y)",
    "P:PacketSystem(X,I)",
    "T:End(X)"
  ],
  "returns": "TransportedPacketOperator(U,P,T)"
}
```

Packet exact sequence for an invariant subspace

PSC-I-T-PACKET-EXACT-SEQUENCE

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "K:InvariantSubspace(P)"
  ],
  "returns": "PacketwiseShortExactSequence(K,X,X/K)"
}
```

Packet descent through an invariant quotient

PSC-I-T-QUOTIENT-PACKET-DESCENT

```
{
  "params": [
    "P:PacketSystem(X,I)",
    "K:InvariantSubspace(P)"
  ],
  "returns": "PacketSystem(X/K,I)"
}
```

Naturality of generated hulls and transfer blocks

PSC-I-T-NATURALITY

```
{
  "params": [
    "Phi:PacketCompatibleOperatorMorphism",
    "S:SeedSet"
  ],
```

```
  "returns": "NaturalityCertificate(Phi,S)"
}
```

Realization functor and preservation profile

PSC-I-D-REALIZATION-FUNCTOR

```
{
  "params": [
    "F:AdditiveFunctor(C,D)",
    "profile:PreservationProfile"
  ],
  "returns": "RealizationFunctor(C,D,profile)"
}
```

Equational realization principle

PSC-I-T-REALIZATION-EXPORT

```
{
  "params": [
    "Theta:TheoremRecord",
    "F:RealizationFunctor"
  ],
  "requires": [
    "operations_used(Theta)\u2286verified_preservation(F)",
    "target_hypotheses_verified"
  ],
  "returns": "TransportCertificate(Theta,F)"
}
```

Semantic realization criterion

PSC-I-D-SEMANTIC-REALIZATION

```
{
  "params": [
    "Theta:TheoremRecord",
    "F:RealizationFunctor"
  ],
  "returns": "SemanticRealizationCriterion(Theta,F)"
}
```

C Complete action registry

The registry below contains all 37 action names. “Implemented” describes the runtime dispatcher. It does not assert that all hypotheses of every linked source record are automatically validated. Record associations are links for inspection.

AGGREGATE_PACKETS

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-L-PACKET-AGGREGATION

ANALYZE_PACKETS

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-C-PACKET-UNIVERSAL-PROPERTY; PSC-I-C-PACKETWISE-EQUALITY; PSC-I-D-ANALYSIS-SYNTHESIS; PSC-I-T-EXACT-PACKET-COORDINATES

ASSEMBLE_BLOCK_ALGEBRA

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-DIMENSION-COUNT; PSC-I-T-BLOCK-ALGEBRA

BLOCKIFY_OPERATOR

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-D-TRANSFER-BLOCK; PSC-I-T-OPERATOR-RECONSTRUCTION

BUILD_SEED_HULL

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-D-GENERATED-HULL; PSC-I-D-SEED-EVALUATION

BUILD_SUPPORT_ALGEBRA

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-SUPPORT-PATTERN-STAR; PSC-I-T-PREORDER-ALGEBRA-DECOMPOSITION; PSC-I-T-SUPPORT-PATTERN-ALGEBRA

BUILD_TRANSFER_GRAPH

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-TRANSFER-GRAPH

CHANGE_PACKET_COORDINATES

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-T-COORDINATE-TRANSPORT

CHECK_REALIZATION_PROFILE

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_SEMANTIC_GATE

Records. PSC-I-D-REALIZATION-FUNCTOR; PSC-I-D-SEMANTIC-REALIZATION

COMBINE_SEED_SYSTEMS

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-P-SHARED-DIRECT-SUM; PSC-I-P-TENSOR-BOUND; PSC-I-T-EXACT-SEQUENCE-BOUNDS; PSC-I-T-INDEPENDENT-DIRECT-SUM

COMPLETE_PACKET_SPLIT

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-COMPLETION-PACKET-SPLIT; PSC-I-T-COUNTABLE-UNCONDITIONAL-PACKETS

COMPLETE_TOPOLOGICAL_REALIZATION

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-TOPOLOGICAL-PACKET-EXACT-SEQUENCE; PSC-I-T-DENSE-REALIZATION; PSC-I-T-QUOTIENT-TOPOLOGY-COMPLETION

COMPOSE_TRANSFER_BLOCKS

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-C-TENSOR-TRANSFER-BLOCKS; PSC-I-T-BLOCK-MULTIPLICATION

COMPUTE_PACKET_PURE_RANK

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-SOURCE-COMPONENT-COUNT; PSC-I-D-DIRECTED-SEED-RANK; PSC-I-P-PACKET-PURE-RANK-BOUNDS; PSC-I-T-SOURCE-COMPONENT-SEED-RANK

COMPUTE_REACHABILITY

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-P-REACHABILITY-OBSTRUCTION; PSC-I-T-ALGEBRA-REACHABILITY-SUPPORT

COMPUTE_SEED_RANK

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-ALGEBRAICALLY-CLOSED-SINGLE-OPERATOR; PSC-I-C-INVARIANT-FACTOR-COUNT; PSC-I-D-DIRECTED-SEED-RANK; PSC-I-D-SEED-RANK; PSC-I-P-CYCLIC-CHARACTERIZATION; PSC-I-T-MODULE-TOP; PSC-I-T-SEMISIMPLE-SEED-RANK; PSC-I-T-SINGLE-OPERATOR; PSC-I-T-SOURCE-COMPONENT-SEED-RANK

DECOMPOSE_PRIMARY

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-ALGEBRAICALLY-CLOSED-SINGLE-OPERATOR; PSC-I-C-CYCLIC-MATRIX; PSC-I-C-INVARIANT-FACTOR-COUNT; PSC-I-T-SINGLE-OPERATOR

DESCEND_PACKET_QUOTIENT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-C-QUOTIENT-PACKET-DESCENT; PSC-I-T-QUOTIENT-PACKET-DESCENT

DUALIZE_PACKETS

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-P-DUAL-PACKET-SYSTEM

EVALUATE_PACKET_CONDITIONING

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-PACKET-CONDITION-NUMBER; PSC-I-P-CONDITIONED-COORDINATES

EXPAND_OPERATOR_PATHS

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-C-BLOCK-POWERS

EXPORT_PSCI_INTERFACE

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-EXPORT

EXTRACT_PACKET_COMMUTANT

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-P-BLOCK-DIAGONAL-COMMUTANT

FACTOR_ACTION

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-P-ACTION-FACTORIZATION

LOWER_INTERTWINER

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-T-BLOCK-INTERTWINING

PRESENT_AS_FREE_QUOTIENT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-MINIMAL-FREE-COVER; PSC-I-C-ONE-SEED-PRESENTATION; PSC-I-T-FREE-PRESENTATION

PROPAGATE_PACKET_SUPPORT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-C-PACKET-SUPPORT-PROPAGATION

PUSH_GENERATORS_THROUGH_QUOTIENT

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-P-SCALAR-QUOTIENT-WITNESS; PSC-I-T-QUOTIENT-MONOTONICITY

REALIZE_GRAMMAR

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-COLLAPSED-REALIZATION; PSC-I-T-TYPED-MODULE-EQUIVALENCE

REDUCE_TO_SEMISIMPLE_TOP

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-LOCAL-ALGEBRA-FORMULA; PSC-I-T-MODULE-TOP; PSC-I-T-SEMISIMPLE-SEED-RANK

RESOLVE_PACKET_EXACT_SEQUENCE

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-C-TOPOLOGICAL-PACKET-EXACT-SEQUENCE; PSC-I-T-PACKET-EXACT-SEQUENCE

TENSOR_PACKETS

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY

Records. PSC-I-T-TENSOR-PACKET-SYSTEM

TRANSPORT_MORPHISM

Runtime implemented. No

Modes. EXACT_FIXTURE_ONLY, PLANNER_ONLY

Records. PSC-I-D-PACKET-MORPHISM; PSC-I-D-SEED-MORPHISM; PSC-I-P-SEED-SYSTEM-CATEGORY; PSC-I-T-NATURALITY

TRANSPORT_SEED_RANK

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-C-ISOMORPHISM-INVARIANCE; PSC-I-P-SCALAR-EXTENSION-MONOTONICITY

TRANSPORT_THEOREM

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_SEMANTIC_GATE

Records. PSC-I-P-REALIZATION-EXPORT; PSC-I-T-REALIZATION-EXPORT

VALIDATE_PACKET_SYSTEM

Runtime implemented. Yes

Modes. RUNTIME_EXECUTABLE_EXACT_FINITE

Records. PSC-I-D-PACKET-SYSTEM

VERIFY_PROPAGATION_HYPOTHESES

Runtime implemented. No

Modes. PLANNER_ONLY

Records. PSC-I-D-EDGE-SURJECTIVE; PSC-I-D-PACKET-PROPAGATING

D Operation and hypothesis vocabulary

The runtime ontology separates structural operations from hypothesis classes. These names form a controlled vocabulary for profiles and source-derived annotations. They are not a complete formal semantics for every mathematical construction named.

Name	Class
`identity`	structural
`zero`	structural
`finite_sum`	structural
`additive_inverse`	structural
`composition`	structural
`scalar_action`	structural
`direct_sum`	structural
`finite_product`	structural
`biproduct`	structural
`kernel`	structural
`cokernel`	structural
`image`	structural
`quotient`	structural
`tensor_product`	structural
`dual`	structural
`packet_idempotents`	structural
`transfer_blocks`	structural
`support_relation`	structural
`path_composition`	structural
`polynomial_action`	structural
`jacobson_radical`	structural
`semisimple_decomposition`	structural
`primary_decomposition`	structural
`universal_property`	structural
`functoriality`	structural
`exact_sequence`	structural
`closure`	structural
`completion`	structural
`topology`	structural
`norm`	structural
`adjoint`	structural
`dimension`	structural
`finite_indexing`	hypothesis
`surjectivity`	hypothesis
`injectivity`	hypothesis
`isomorphism`	hypothesis
`continuity`	hypothesis
`boundedness`	hypothesis
`closedness`	hypothesis
`density`	hypothesis
`orthogonality`	hypothesis
`self_adjointness`	hypothesis
`positivity`	hypothesis
`integrality`	hypothesis

Name	Class
unconditional_convergence	hypothesis
hausdorff	hypothesis
complete_field	hypothesis
invariant_subspace	hypothesis
commutation	hypothesis
transitivity	hypothesis
edge_surjectivity	hypothesis
packet_propagation	hypothesis
nonzero_packet_spaces	hypothesis

E Runtime Lean record map

This is the eight-entry mapping attached to the archived runtime. The evidence descriptions and source-match categories are reproduced from that map. A fresh build performed elsewhere must be documented separately.

Finite packet system

PSC-I-D-PACKET-SYSTEM

Source match. `structural_exact`

Stored certificate status. `compiled_in_verified_tranche`

Declarations. `Komorebi.Foundation.LinearProjectorFamily`

In the runtime formalization, ProjectorCore.lean supplies structure axioms for idempotence, pairwise annihilation, and completeness.

Packet analysis and synthesis

PSC-I-D-ANALYSIS-SYNTHESIS

Source match. `stronger_typed_realization`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.LinearProjectorFamily.PacketSpace`; `Komorebi.Foundation.LinearProjectorFamily.rangeAnalysis`; `Komorebi.Foundation.LinearProjectorFamily.rangeSynthesis`

ExactCoordinates.lean defines the packet space as the product of projector ranges, so both transforms carry range certificates.

Exact packet-coordinate isomorphism

PSC-I-T-EXACT-PACKET-COORDINATES

Source match. `stronger_typed_realization`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.LinearProjectorFamily.exactPacketCoordinates`

Explicitly listed as Kernel accepted in VERIFICATION_REPORT.md; proves both inverse laws for range-restricted packet coordinates.

Packetwise equality

PSC-I-C-PACKETWISE-EQUALITY

Source match. `one_direction_plus_trivial_reverse`

Stored certificate status. `compiled_in_verified_tranche`

Declarations. `Komorebi.Foundation.LinearProjectorFamily.eq_of_projectors_eq`

The nontrivial direction is proved in ProjectorCore.lean; the reverse direction follows by congruence from equality.

Transfer block

PSC-I-D-TRANSFER-BLOCK

Source match. `generalized_domain_codomain`

Stored certificate status. `compiled_in_verified_tranche`

Declarations. `Komorebi.Foundation.transferBlock`

Definition allows distinct source and target packet systems; the PSC-I endomorphism case is a specialization.

Exact operator reconstruction

PSC-I-T-OPERATOR-RECONSTRUCTION

Source match. `generalized_domain_codomain`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.transferBlock_reconstruction`

Explicitly listed as Kernel accepted; reconstructs any linear map from all source-target blocks using completeness of both packet families.

Packet matrix multiplication

PSC-I-T-BLOCK-MULTIPLICATION

Source match. `generalized_composition_form`

Stored certificate status. `kernel_accepted`

Declarations. `Komorebi.Foundation.transferBlock_comp`

Explicitly listed as Kernel accepted; inserts the complete middle packet family and yields the local composition sum.

Support-pattern algebra criterion

PSC-I-T-SUPPORT-PATTERN-ALGEBRA

Source match. `partial_sufficient_direction`

Stored certificate status. `kernel_accepted_related`

Declarations. `Komorebi.Algebra.MatrixSupported.mul`

Verified tranche proves multiplicative closure for transitive support relations; it is related evidence, not a source-equivalent certificate for every direction of the PSC-I criterion.

F Preserved verification receipt

The following is the recorded successful runtime verification run. The detailed exact-fixture output, command lines, interpreter versions, earlier missing-dependency explanation, and frozen-byte audit are retained in the build results.

```
ENTRYPOINT_VERSION=PASS
RECORD_COUNT=PASS
DEPENDENCY_ACYCLIC=PASS
DEPENDENCIES_RESOLVE=PASS
AST_COVERAGE=PASS
ACTION_REGISTRY=PASS
SOURCE_HASH=PASS
API_STATUS=PASS
API_RECORD=PASS
API_DEPS=PASS
API_AST=PASS
EXECUTE_BLOCKIFY=PASS
EXECUTE_PACKET_VALIDATE=PASS
TRANSPORT_GATE_STRUCTURED=PASS
EXACT_FIXTURE_REPLAY=PASS
PSC_I_MACHINE_V1_RELEASE_GATE=PASS
```

Interpreter provenance

```
3.12.14 (main, Aug 25 2026, 14:00:49) [Clang 22.1.3 ]
SymPy=1.14.0
```

Source and package preservation

The companion's integrity audit compared each of the 65 original archived files with the copied payload. Every comparison passed. Generated Python bytecode caches, if any, are not part of the archived source and are excluded from the delivered package.

Catalogue provenance. The 108 source statements and 69 proof bodies are generated from the archived mathematical inventory. Their printed hashes identify that source wording. The catalogue removes outer statement environments and translates cross-references into links to stable records. Formal acceptance is documented separately in the verification index.